

The Economics of Coding Agents: Calibrating the Cost-Quality Frontier

Aninda Ray*

Abstract

Coding-agent deployments overpay in two ways. They grind on tasks that no configuration at their model tier will ever resolve, and they buy quality with extra turns that do not pay. We present **Code-Capsules**, a configuration framework for coding-agent execution that minimizes cost without degrading quality. Its first contribution is a stopping signal. When two diverse configurations both fail the same task at one model tier, that task is hard at that tier. This inverts self-consistency, the usual practice of trusting an answer when independent runs agree: here, agreement on failure flags a task as hard. The obvious use is to stop early and save money, and we report, as a first-class result, that this honest deployment test fails. Within a single model tier the signal adds no resolves over its best configuration and costs more, because a second attempt roughly doubles spend for little gain. The flagged tasks are mostly recoverable at a stronger tier rather than genuinely doomed, so the signal pays off instead at the tier boundary. A lightweight test-based check, rerunning the existing regression suite, recovers most of the wrongly-abandoned resolves at no added cost. Escalating the flagged set one tier up then resolves nearly all of what remains, at the stronger model’s cost: a value-of-resolve decision, not a free default. The second contribution is a calibrated *menu* of operating points on one common cost model, from a cost-minimum configuration to a strong-tier quality maximum that out-resolves the unbounded mid-tier baseline at lower cost per resolve (\$0.56 versus \$0.63). The menu replicates on a held-out instance split. The framework ships behind eight extension primitives and a vendor-agnostic model runtime (Anthropic, OpenAI, and Google Gemini), so deployers can re-derive the menu on their own workload. We calibrate the shipped defaults on SWE-bench Lite (bug fixing), HumanEval [9], and MBPP [4] across four model tiers spanning Anthropic and OpenAI. A plan-before-acting scaffold helps every model we tried, while a critique-and-retry scaffold helps only some. Code-Capsules extends the predecessor framework Agent Capsules [30] from multi-agent research pipelines to single-session coding agents; code, calibrated defaults, and per-cell evaluation data are publicly available.

Keywords

coding agents, large language models, SWE-bench, agent harnesses, autonomous bug fixing, extensible runtimes

1 Introduction

A coding agent is a program that performs software-engineering tasks on its own: completing functions from specifications, refactoring code, generating tests, or fixing bugs in an existing repository. The agent reads files, navigates the directory tree, edits source code, runs the test suite, and iterates until the tests pass or it gives up. Current language models paired with an agent harness now

resolve a substantial fraction of such tasks without human intervention, including the real GitHub bug-fix issues collected in SWE-bench [18, 19], a benchmark drawn from twelve popular Python projects. The framework we present is agnostic to the task class; we calibrate its shipped defaults on three publicly available benchmarks (SWE-bench Lite for bug fixing, HumanEval [9] for single-function completion, and MBPP [4] for basic programming problems), with bug fixing carrying the headline cost-quality findings because the per-attempt cost variance is sharpest there. HumanEval and MBPP serve as cross-vendor parity checks on saturated single-completion workloads. As coding agents move from research artifacts into production deployments, the cost per attempt has become a practical concern. A single attempt on a hard issue can issue dozens of model calls and consume hundreds of thousands of tokens of input, and the deployer’s bill scales with both.

How an agent is configured changes its cost substantially. The model running the agent, the prompt that frames the task, the turn budget the deployer sets, and the auxiliary information given to the model at the start of a session all matter. Different combinations produce different cost-quality outcomes on the same workload, and a single configuration choice rarely answers for an entire deployment. A deployment overpays in two distinct ways: by grinding on instances that no configuration at its model tier will resolve, and by buying quality with extra turns that do not pay. This paper is about controlling both.

We present **Code-Capsules**, an extensible framework for coding-agent execution that treats configuration as a selection problem over a catalog of mechanisms. Each mechanism is a separate, swappable decision point in how the agent handles a task, so a deployer can change one behavior without rewriting the orchestration. The framework’s surface is a set of eight extension primitives: a model-runtime abstraction (`ModelClient`), a workload classifier, a routing strategy, seven shipped execution variants, signals, quality gates, cascade triggers, and cost models. Each primitive is a Python Protocol with shipped defaults and a user-overridable hook; the shipped defaults are calibrated on SWE-bench Lite, HumanEval [9], and MBPP [4] across the Anthropic 4.X family (Haiku 4.5, Sonnet 4.6, Opus 4.7). The model-runtime primitive additionally ships working implementations for OpenAI gpt-5-codex (via the Responses API [28]) and Google Gemini 2.5-flash [12]. The extension surface is described in Section 3.

We make six contributions.

- (1) A stopping-signal mechanism and its honest deployment test (Section 9). The signal is this. When two diverse configurations both fail the same instance at one model tier, that instance is hard at that tier: the best in-family pair fires on 22 instances and abandons only 2 that another configuration would have resolved. The usual self-consistency heuristic trusts an answer when several runs *agree*; here we read the same agreement on *failure* as evidence the

*Correspondence: research@anindaray.com. Code: github.com/array-17/code-capsules (tag v1.0-arxiv).

task is hard at the current tier. We then deployed the signal as a single-tier controller that runs both configurations and abandons the instance when they agree, integration-tested leak-free on all 300 SWE-bench Lite instances. In that form the mechanism is *retired*: it adds zero resolves over its own best member, and a single-tier abandon destroys real resolves at higher cost per resolve, a first-class negative (Section 13.5). What survives deployment is the gate form. A cheap regression-suite gate ships a candidate when its execution check or the repository’s regression suite passes instead of abandoning it, recovering most of the wrongly-abandoned resolves at no added cost. The flagged instances are mostly recoverable one tier up rather than doomed, so the signal pays off at the tier boundary, where escalating the flagged set to a stronger model resolves nearly all of it (Sections 9.6, 9.8).

- (2) A calibrated cost-quality menu (Sections 4, 8): five operating points on one modeled cost surface, from a relevance-ranker configuration at \$0.34 per resolved instance to a strong-tier ceiling at \$0.66, in which the Opus twenty-turn implicit cell (128/150 at \$0.56 per resolve) resolves more than the unbounded Sonnet baseline (111/150 at \$0.63) while costing less per resolve. Each entry’s role is measured, the menu replicates on a held-out split, and a matched re-run of an external baseline (Section 11) anchors it: the balanced entry resolves more, at 13.8% lower cost per resolve, a significant quality edge ($p = 0.0055$).
- (3) An eight-primitive extension surface (Section 3): how the mechanism and the menu port. Users override any layer of the agent loop, from the execution strategy down to the model-runtime abstraction; the surface includes a vendor-agnostic model-runtime primitive with working implementations for Anthropic Claude Code [3], OpenAI Chat Completions and Responses, and Google Gemini, enabling cross-vendor evaluation on the same orchestration code. Six of the eight primitives are exercised by the SWE-bench Lite headline evaluation; the routing strategy and workload classifier are inactive on this monomorphic benchmark (a single workload class) and exercise on mixed-workload inputs not in scope for this paper (Section 6.3).
- (4) A cross-vendor measurement at $n=150$ on SWE-bench Lite across Anthropic Sonnet, OpenAI gpt-5-codex, and the Anthropic Opus and Haiku tiers. Both the data-axis scaffold (plan-then-execute) and the iteration-axis scaffold (two-pass critique) lift pass rate on both vendors; under the always-run configuration two-pass adds +3 pp on Sonnet and +12 pp on codex over plan-then-execute, though a ~44% empty-patch failure rate on codex caps its ceiling about 21 points below Sonnet’s.
- (5) A workload-reachability finding with a shipped tail disclosure: across every shipped variant on every tier evaluated, the portfolio resolves 148/150 (98.7%) of the SWE-bench Lite first 150 under reference-patch-validated scoring (an oracle union over configurations, oracle-assisted), leaving only 2 matplotlib instances unreachable by any configuration. There is no large irreducible failure set on this split; the

shipped `policy.yaml` instead scopes the genuinely budget-insensitive instance tail (Section 12) so deployers do not pay unbounded turns against it.

- (6) A leak-free evaluation protocol and a composition discipline (Sections 13.6, 13.5). A control-flow read of a benchmark-only verdict (using it to decide when to stop or retry, never to generate a patch) biases cost down and resolve counts up while leaving every patch legitimate; and a composed controller can resolve no more than its own best single member while costing far more and destroying real resolves through its stopping rule. Both traps generalize beyond this framework. The protocol we ship closes the first: deployable signals (reproduction scripts, regression suites, patch presence) drive every control decision, and the benchmark verdict only scores. The second is caught by a composition rule: a controller must beat its own best member, leak-free and end-to-end, before it ships.

1.1 Extending Agent Capsules to single-session coding agents

Code-Capsules extends Agent Capsules [30] in three specific ways. First, the **mechanism-portfolio methodology**: selecting from a calibrated catalog of mechanisms rather than committing to a single approach. Agent Capsules introduced this for multi-agent pipelines (a composition score chooses fine versus compound execution; an escalation ladder restores quality on regression); Code-Capsules adopts the same architectural intent for single-session coding agents (a workload classifier chooses among variants; a cascade trigger re-attempts on quality-gate failure). Second, the **extension-surface pattern**: Protocol-based primitives with name-keyed registries and shipped defaults validated by the framework’s own evaluations. Agent Capsules shipped one such Protocol (QualityEvaluator); Code-Capsules generalizes the pattern to eight, every load-bearing decision point in the agent loop exposing the same shape. Third, the **calibration discipline**: per-tier deployment defaults selected from measured Pareto frontiers rather than guessed. Agent Capsules ships sensitivity presets (aggressive/balanced/conservative) anchored on measurement; Code-Capsules ships per-(workload, tier) menu tables (`cost_min/balanced/quality/quality_max/ceiling`) on the same principle.

The two systems do not share runtime or policy schemas. Single-session coding agents have a different agent topology than multi-agent research pipelines, so the implementations diverge: execution variants that change the structure of the agent loop rather than composition scores over agent dependency graphs, post-attempt Docker validation rather than gate-over-output-structure, an eight-Protocol extension surface rather than a single injection point. The architectural intent and the measurement-first calibration carry over. Section 14.1 maps each Agent Capsules claim we extend or refine.

2 Background and Motivation

2.1 Cost structure of an agent session

The cost of a coding-agent session is the sum of its per-turn LLM-call costs. Each call’s input is the running conversation history

(system prompt, prior user and assistant messages, prior tool results) and the output is the next assistant message plus any tool-call arguments. The agent executes as a loop: on each iteration (a *turn*), it receives the conversation, decides what to do next, and either responds with text or invokes a tool from a fixed surface. In our harness the surface comprises file-level operations (`Read`, `Write`, `Edit`) and a shell-execution tool (`Bash`); each invocation is a structured tool call that the model emits in its response and the framework executes against the worktree. The tool’s result is appended to the history and the loop repeats until the agent believes the patch is complete, gives up, or hits the deployer’s turn budget. A typical successful SWE-bench Lite session uses ten to forty turns; a failed session often runs to the budget.

Because the conversation grows monotonically, later turns send proportionally larger input than earlier ones. Two structural facts dominate cost analysis. First, the tenth turn is far more expensive than the first: per-call input grows roughly linearly with turn count, so total session cost grows roughly quadratically in turn count when no caching is in effect. Second, vendor prompt caches [2, 12, 27] discount cached input tokens to roughly ten percent of the fresh rate. Because each turn re-sends most of the prior conversation, cache hit rates on agent loops are structurally high after the first turn, and the marginal cost of a follow-up turn is substantially smaller than the per-call cost would suggest. Naive cost accounting that multiplies token counts by public pricing overstates real billed cost on agent workloads; Section 5.6 documents how Code-Capsules accounts for caching.

2.2 SWE-bench Lite and the configuration space

SWE-bench [19] is a benchmark of 2,294 real GitHub issues drawn from twelve large Python repositories: `django`, `sympy`, `matplotlib`, `astropy`, `scikit-learn`, `sphinx`, `pytest`, `requests`, `flask`, `xarray`, `pylint`, and `seaborn`. Each instance specifies the repository state at issue-open time, the natural-language bug report, and a hidden *test patch* that the proposed fix must satisfy under Docker evaluation. SWE-bench Lite [18] is a curated 300-instance subset selected for evaluator reliability. Our evaluations use the first 30, 50, or 150 instances of SWE-bench Lite; sample sizes per cell are disclosed in Section 5.4.

Code-Capsules treats agent execution as a function of seven configuration knobs: the tunable dimensions of a single run. These are a different organizing axis from the eight extension primitives of Section 3, which are the Protocol types a deployer re-implements. The **turn budget** bounds how many turns the agent receives. **Budget signaling** controls whether that budget is communicated to the model in the system prompt. The **variant** selects the high-level execution strategy: signaled budget, plan-then-execute, two-pass critique, or one of three other shipped options (Sections 6 through 7). The **relevance ranker**, optionally, prepends a list of likely-relevant files to the system prompt. The **model tier** selects among Haiku 4.5, Sonnet 4.6, and Opus 4.7. The **model runtime** (the `ModelClient` primitive) selects the vendor and transport: Claude Code CLI, OpenAI Chat Completions, OpenAI Responses (required for `gpt-5-codex`), or Gemini. Finally, the **cascade trigger** controls whether and when to escalate to a stronger variant after a failed first attempt. The framework ships defaults for each knob; per-workload calibration lives in `policy.yaml`.

2.3 Relationship to Agent Capsules

Agent Capsules [30] characterized the cost-quality tradeoff in multi-agent pipelines that synthesize structured outputs, such as a five-agent due-diligence pipeline that produces a structured company report. The mechanisms it introduced operate on the structure of agent declarations: which agents share context, which agents may merge, which outputs need quality review. Its composition score selects when to consolidate multiple agents into a single LLM call, its quality gate blocks unsafe consolidations, and its escalation ladder recovers quality when consolidation degrades output structure.

Code-Capsules operates on the structure of a single agent’s session instead: which turns are spent on which sub-task, which files are made visible to the model, when to re-attempt after a failed first patch. The mechanism surfaces are different because the workloads have different cost structure, but the architectural intent is the same: ship a measurement-driven mechanism catalog with calibrated defaults, and provide an extension API for users whose workload, vendor, or quality target differs from the shipped calibration.

The two papers also differ in the empirical pressure motivating them. Agent Capsules addresses the cost of repeated system-prompt ingestion across multiple agent calls in a single pipeline run. Code-Capsules addresses the per-turn input growth of an agent’s conversation across a single session, which can reach hundreds of thousands of tokens for hard SWE-bench Lite instances under modern harnesses. The per-workload calibration question (which mechanism does best, at what cost, on which model tier, under which vendor) has substantive answers that vary by workload class, and the framework reports them per cell rather than prescribing a single default.

3 Extension surface

Figure 1 gives the framework at a glance. The framework contribution is the eight-Protocol surface, not any individual mechanism. Every load-bearing decision point in the agent execution loop is the same kind of object: a Python Protocol with shipped default implementations, deployer override by name in `policy.yaml`, and registry-based custom implementations. The per-cell evaluations in this paper validate the shipped defaults, not the primitives themselves.

This generalizes a narrower pattern. Agent Capsules [30] exposed one user-overrideable injection point, the `QualityEvaluator` Protocol, with a per-group policy override and a pluggable state backend. Selection among shipped mechanisms (composition score, escalation ladder, quality gate, budgeted hint) happened through configuration. The mechanisms themselves were not framed as user-replaceable primitives. Code-Capsules turns every such decision point into an extension primitive of the same kind.

Each Protocol defines a single decision point in the agent execution loop and each ships with default implementations that the evaluations in this paper validate. Users select among shipped implementations by name in `policy.yaml`, or register their own through a single-line registry call. The extension surface is the same one the framework uses to calibrate its shipped defaults; there are no privileged internal primitives.

Table 1 lists the eight primitives, the decision each one makes, the shipped implementations available out of the box, and the section

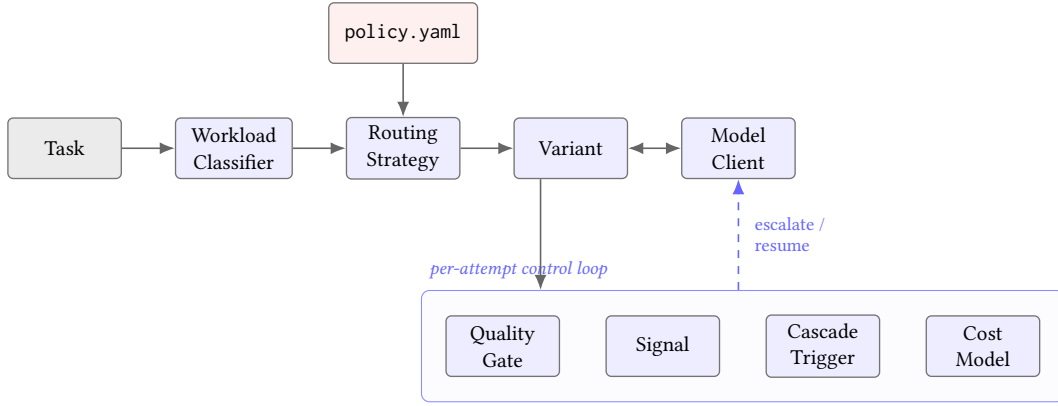


Figure 1: The framework at a glance, showing all eight extension primitives (blue). A task (a GitHub issue plus a checked-out worktree) enters at left. The workload classifier assigns a class label; the routing strategy maps that label to a variant configuration drawn from `policy.yaml`; the variant drives the agent through the model client, a vendor-agnostic model-runtime abstraction with shipped Anthropic, OpenAI, and Google implementations, and receives the attempt back (the double-headed arrow is that call-and-return round-trip). After the attempt, the per-attempt control loop scores it. The quality gate decides whether the patch resolves the issue. A signal reports session-level stall and progress detectors. The cascade trigger reads the signal together with an internal execution verifier grade and chooses ship, escalate, or hand back. The cost model prices the attempt. On escalate, the model client is resumed with a larger turn budget or a stronger model tier. The shipped cascade-trigger default is the regression-gated form of §9.6: a candidate ships when its execution check or the repository’s regression suite passes, and diverse agreement on failure marks the instance for escalation or hand-back rather than silent single-tier abandonment. Task (gray) is the input and `policy.yaml` (red) the calibrated per-cell configuration; neither is a primitive.

in this paper where the primitive is evaluated. The remainder of this section is a brief walkthrough of how the primitives compose in a single agent invocation.

An invocation begins when a deployer calls the framework’s runner with a task descriptor (in our SWE-bench harness, the issue text plus the path to a clean repository worktree). The **workload classifier** assigns a class label to the task. The **routing strategy** consumes the label and selects an execution variant, either by static rule (the default policy maps a label to a single variant) or by cascade (a sequence of variants applied in order until one succeeds). The selected **variant** drives the agent loop: it issues prompts to the **model runtime**, which in turn adapts the framework’s per-turn tool-call records to the vendor’s native tool-calling API, executes tool calls in the worktree, and accumulates per-turn invocation records.

While the variant is running, **signals** observe the session state. Each shipped signal returns a structured observation (the turn budget is nearly exhausted; the agent has been re-reading the same file; the latest test invocation failed with a particular traceback) that the variant can consult to make decisions within the session. When the variant ends a session, the **quality gate** evaluates the attempt under Docker for SWE-bench, under pytest for HumanEval and MBPP, or under a user-supplied checker. The **cascade trigger** consumes the gate’s verdict and decides whether the routing strategy should re-attempt the task with a different variant. When the quality gate cannot confirm a patch, do we ship it, try again, or hand it back? Two shipped cascade triggers answer that question by implementing the stopping-gate defaults of §9. `verifier_gate` re-runs the candidate patch against a derived execution check and re-attempts

only when that check fails. `diverse_agreement` consumes the verdicts of two diverse same-tier configurations; its shipped default is the regression-gated hybrid form (§9.6): a candidate ships when its execution check *or* the repository’s regression suite passes, and when both configurations agree on failure the instance is marked for opt-in model-tier escalation or hand-back to the caller. Silent single-tier abandonment is not a shipped default; its integration test retired it (§13.5). The **cost model** computes per-attempt and per-session cost from the accumulated invocation records, honoring per-vendor prompt-cache discounts (Section 5.6).

The extension pattern is symmetric across primitives: each one is a Python Protocol type with a name-keyed registry. To add a new primitive implementation, a user defines a class conforming to the Protocol, implements its methods, and registers it through a single registry call. The shipped implementations are themselves the result of this pattern, registered the same way; no internal API is reserved. Sections 6 through 10 report measurements under the shipped defaults; users in other domains override individual primitives and re-derive their cost-quality frontier on their own data.

Three of these primitives carry the deployable system (Figure 2), and the rest of the paper refers to them by role. The **solvers** (the Variant implementations) are the patch generators; the machine selects among a diverse pair of them and escalates across model tiers. The **reproduction verifier** (the QualityGate, in its generated fail-on-base reproduction and regression-suite forms) is the deployable proxy for the benchmark verdict: it grades a candidate without consulting the held-out tests and drives every ship decision, the *quality lever*. The **governor** (the CascadeTrigger) reads those

Table 1: The eight extension primitives shipped by Code-Capsules. Each primitive is a Python Protocol; the shipped implementation names are the values a deployer selects in `policy.yaml`.

Primitive	Decision it makes	Shipped implementations	Evaluated in
ModelClient	Agent runtime	claude_cli, openai_api, openai_responses, gemini_api	\$10
WorkloadClassifier	Task class	generic_prompt, benchmark_saturated, yaml_mapping	\$6.3
RoutingStrategy	Variant selection	rule_based, cascade, default	\$6.3
Variant	Execution strategy	7 shipped (Sections 6, 7)	\$7
Signal	Session-state observation	cap_pressure, file_thrash, test_failure, traceback	\$6.4
QualityGate	Attempt-success verdict	docker_eval, binary_tests, python_ast, always_pass	\$5.2
CascadeTrigger	Re-attempt decision	heuristic, always_escalate, never_escalate, diverse_agreement, verifier_gate	\$7.1, \$9
CostModel	Per-attempt cost	anthropic_public, openai_public, gemini_public	\$5.6

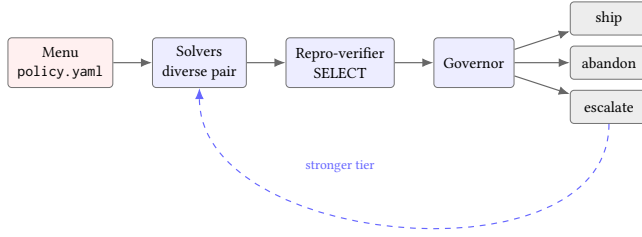


Figure 2: The deployable anatomy: the three load-bearing components. The calibrated *menu* (`policy.yaml`) selects a solver configuration for the workload; the *solvers* (a diverse pair) produce candidate patches; the *repro-verifier* keeps the patch a generated fail-on-base reproduction confirms (the quality lever); and the *governor* reads the verifier and regression-suite signals and chooses ship, abandon, or escalate to a stronger model tier (the cost lever). This is the same machine as Figure 1, viewed by deployable component rather than by extension primitive: the solvers are Variant implementations, the repro-verifier is the QualityGate, and the governor is the CascadeTrigger.

grades and decides whether to ship, abandon, or escalate, the *cost* lever. The calibrated menu (Section 4) selects the solver; the verifier and governor together are the stopping mechanism (Section 9); and the negative results (Section 13) report what happens when the three are composed into a single-tier controller.

4 Configuration and policy

A deployer interacts with the framework through a single YAML configuration file (`policy.yaml`) and a runner constructor that consumes it. This section describes the configuration surface, the rule by which shipped defaults are derived from per-cell measurements, and the override pattern for users who need to deviate from the shipped recommendation.

4.1 Configuration in YAML

The configuration file has two top-level blocks. The routing block configures the workload classifier, routing strategy, and quality-gate thresholds (framework-level knobs that apply to every deployment). The deployment_defaults block is a per-tier, per-workload recommendation table: one entry per (*workload-class*, *model-tier*)

Table 2: The shipped cost-quality menu for hard agent workloads (SWE-bench Lite first 150, cache-aware accounting, one modeled cost surface). Columns \$/att. and \$/res. are dollars per attempt and dollars per resolved instance, respectively. Every entry resolves more per dollar than the external baseline of Section 11 (\$0.90 per resolve); the `quality_max` entry additionally resolves more than the unbounded Sonnet baseline (the quality row, not the Opus ceiling row) while costing less per resolve. Absolute resolve counts carry the target-test-identifier disclosure of Section 15, a symmetric upward bias across all cells. Operators select an entry by name through `policy_for(knee=...)`; the framework returns the typed policy ready to drive a runner.

Menu entry	Variant (tier)	Resolved	\$/att.	\$/res.
cost_min	Sonnet, relevance ranker ($b=10$)	80/150	0.183	0.34
balanced	Sonnet, two-pass critique ($10 \rightarrow 25$)	98/150	0.408	0.62
quality	Sonnet, unbounded budget ($b=100$)	111/150	0.466	0.63
quality_max	Opus, implicit budget ($b=20$)	128/150	0.480	0.56
ceiling	Opus, unbounded budget ($b=100$)	138/150	0.604	0.66

combination, with up to five cost-quality menu entries per workload class (the selection rule is described in Section 4.2). Each menu entry is a `CodeCapsulesPolicy`: a typed dataclass whose fields name the variant, mode, turn budget, escalation budgets, and mechanism selections that variant requires. The five shipped menu entries for hard agent workloads are summarised in Table 2; saturated workload classes ship their own entries under the same schema (Section 8.7).

Operators interact with the policy declaratively: name the cell they want via `policy_for`, or construct a policy from scratch with named keyword arguments for deployments outside the shipped calibration. Listing 1 shows both forms; loading the policy directly from a `policy.yaml` calibration table is also supported via `CodeCapsulesPolicy.from_yaml`. The same runner is constructed from any of the three forms with a single call.

Listing 1: Two consumer-facing ways to obtain the balanced policy from Table 2: the shipped preset selector or direct typed construction. Both produce identical policies; both drive the runner identically.

```

from code_capsules import (
    CodeCapsulesPolicy, policy_for)

# Option 1: pull from the shipped calibration table.
policy = policy_for(

```

```

workload="hard_workload",
tier="sonnet",
knee="balanced",
)

# Option 2: construct directly with named kwargs.
policy = CodeCapsulesPolicy(
    variant="two_pass_critique",
    mode="escalating",
    turn_budget=10,
    escalating_start_budget=10,
    escalating_target_budget=25,
    always_escalate=True,
    prompt_variant="two_pass_critique",
    # Stopping gate: ship when the execution check
    # or the regression suite passes; on diverse
    # agreement-on-failure, escalate gate-positive
    # instances or hand back -- never silently
    # abandon at a single tier.
    cascade_trigger="diverse_agreement",
    controller={
        "configs": ["unbounded_budget",
                    "signaled_budget"],
        "tiers": ["sonnet"],
        "min_samples": 2,
        "governor": "hybrid_regok",
        "ship_gate": "repro_or_regok",
        "escalation": {
            "mode": "gate",
            "ladder": [{"tier": "opus",
                        "configs": ["unbounded_budget"],
                        "trigger": "regok_true"}],
        },
    },
)

```

In the controller block above, `regok` abbreviates “regression-suite OK”: the candidate patch leaves the repository’s existing tests green. `hybrid_regok` ships a patch when either the reproduction check or the regression suite passes, and `regok_true` escalates the instances the regression suite still flags. The mechanism and its calibration are in Section 9.6.

A cloud-provider or production deployment typically substitutes one or more primitives via the framework’s registry. An operator with a custom quality gate defines a class conforming to the `QualityGate` Protocol and registers it; the same policy loader picks up the override without code changes elsewhere.

4.2 From per-cell measurements to deployment defaults

The shipped defaults are not arbitrary picks; each is selected from the cross-tier Pareto frontier of Sections 7 and 8 by a fixed rule. For every workload class, the `deployment_defaults` table exposes up to five cost-quality menu entries: `cost_min` (the cheapest configuration that produces non-trivial pass rate), `balanced` (the mid-cost entry, anchored against the external baseline of Section 11), `quality` (the strongest mid-tier configuration), `quality_max` (the strong-tier entry with the best cost per resolve), and `ceiling` (the unbounded-budget strong-tier upper bound). The deployer selects which entry to use from their cost-quality budget; all five are anchored on the same per-cell evaluation pipeline, and each step up the menu holds or improves quality.

For users in workloads outside the shipped calibration, the methodology is reproducible. Run the variant catalog on a representative sample of the workload through the same evaluation harness; compute pass rate and cache-aware per-attempt cost per cell (Section 5.6); select the upper-left Pareto frontier; map cells to the five menu entries as above. The result is a fresh `deployment_defaults` block that slots into a user-supplied `policy.yaml` without code changes.

4.3 Deployment user-story: three operating points

The same knobs map to three deployment profiles that differ in what they optimize. Each profile is one menu entry from Table 2 plus a stopping policy. None of the three runs two configurations on the same task and selects between their patches: that “run-both” controller was retired by its own end-to-end integration test (§13.5), which found it added no net resolves over its best single configuration while costing roughly twice as much.

Individual developer (local, one task at a time). Cost per task dominates. The deployer picks the cheapest menu entry that clears their pass-rate bar (typically `cost_min`), runs that single configuration, and stops there. They do not run a second configuration on the same task: doing so nearly doubles cost but its routing headroom is at most 9 resolves per 150 (an oracle-assisted bound, §9.7), so it rarely earns its cost back for one developer. Tier escalation is left off: its recovery is marginal (§13.1) and a single developer rarely has a task whose resolution is worth several dollars of strong-tier compute.

Cloud or platform operator (CI and batch). Throughput and aggregate cost dominate over any single task. The operator runs one menu entry across the batch (`balanced`) and applies the regression-suite gate (§9.6) to the failures: candidates whose execution check or regression suite passes ship; the rest is escalated to the strong tier, which recovers 21 of the 21 gate-positive and 15 of the 17 gate-negative instances, reaching 85/150 at \$1.58 per resolve (§9.8). Because almost every flagged instance is recoverable one tier up, the regression-suite split is used only to order the escalation queue under a constrained budget, not to decline gate-negative outright; the small unrecovered residue is queued for review (on the held-out split, 37% of gate-negative patch-bearing instances in fact resolve, so a decline-by-default arm needs per-workload justification, §15).

Consumer-product builder (interactive assistant). Latency and per-query cost dominate, and the user is in the loop. The typical query runs a single mid-cost menu entry (`balanced`); running a second configuration or escalating to the strong tier adds latency the interactive setting cannot absorb. The stopping gate serves as a hand-back: when the configuration fails and the regression-suite gate is negative, the product reports that it cannot fix the issue automatically and returns control to the user, rather than spending more latency for marginal recovery.

The decision rule across all three is one sentence. Run one menu entry, never two configurations on the same task; let the regression-suite gate say which failures are worth more spend; and escalate model tier only when a resolve’s value exceeds its escalation cost-per-resolve (the value-of-resolve rule of §13.1).

5 Methods

5.1 Harness and evaluation pipeline

Each SWE-bench Lite instance runs through a two-stage pipeline. The inference stage runs the agent in a local git worktree of the target repository, checked out at the instance’s `base_commit`. On session completion, the harness captures `git diff HEAD` as the candidate patch. The evaluation stage runs in the SWE-bench-provided Docker image for that instance (one image per instance, identified by `instance_id`). It applies the benchmark’s `test_patch` first to update the test files to their post-fix expected form, then applies the candidate patch, then runs the instance’s `FAIL_TO_PASS` and `PASS_TO_PASS` test suites under the testbed Python environment. An instance is reported as resolved if and only if both test suites complete with return code zero (all `FAIL_TO_PASS` tests pass and all `PASS_TO_PASS` tests continue to pass).

The same evaluation pipeline drives all SWE-bench Lite cells reported in this paper. For cross-vendor cells, a second harness wraps the Stage 2 evaluation around the framework’s runner and the `ModelClient` primitive, swapping in OpenAI `gpt-5-codex` (via the Responses API) or Google Gemini without otherwise changing the evaluation surface. HumanEval and MBPP use a separate pytest-based harness: the agent writes a candidate solution into a temporary file and the harness runs the benchmark’s inline test fixture against it. Each benchmark exposes a cross-vendor driver script in the repository’s `examples` directory; both share the same harness pattern and accept any registered `ModelClient` implementation.

5.2 Validation gate

A regression gate in the test suite runs the SWE-bench-provided reference patch (the “gold patch”) for one astropy instance and one Django instance through the Stage 2 evaluation. If either gold patch fails to resolve, the gate fails and the evaluator is presumed broken. The gate is marked as a slow test (it pulls Docker images and takes about a minute end-to-end) and is intended to run before any large-scale evaluation campaign rather than on every commit. Every SWE-bench Lite result reported in this paper was generated under an evaluator that passes this gate.

The underlying principle is general: for any benchmark integration, run the benchmark’s own reference solutions through the evaluator first. A known-correct solution that does not resolve is unambiguous evidence that the evaluator is broken. The pattern ports to any benchmark that ships reference solutions. One concrete consequence: the held-out half of SWE-bench Lite is dominated by sympy, whose canonical specification runs its own `bin/test` runner; every held-out cell in this paper is scored under that canonical runner, validated by reference patches (Section 11).

5.3 Leak-free control protocol

The rule behind every result in this paper is simple. A configuration may only act on information a real deployment would actually have, and never on the answer key. Concretely, every control decision an evaluated configuration makes mid-run (which candidate to select, whether to trigger a second pass, whether to stop, escalate, or hand back) reads only signals a real deployment would have: a generated fail-on-base reproduction script, the repository’s own

regression suite, and patch presence. The benchmark’s held-out test verdict is used to *score* outcomes only, never to drive them. This protocol is load-bearing: all two-pass numbers in this paper run the second pass unconditionally, and all controller results in Section 9 are integration-tested under this protocol end-to-end. The control-channel leakage class it forecloses is characterized in Section 13.6.

5.4 Sample sizes

Sample sizes per evaluation cell were chosen to balance cost-quality information density against API spend. The principle is simple: headline cells get the full first-150 split, cross-tier and cross-vendor probes get 30, and every load-bearing claim is rechecked on a held-out 150. On SWE-bench Lite the headline Anthropic Sonnet 4.6 cells (signaled budget, two-pass critique, plan-then-execute, implicit 40-turn budget, unbounded 100-turn “floor”) use the first 150 instances. The second 150 instances are a measured held-out surface: the unbounded baseline, the stopping controller of Section 9, the two-pass critique variant, and the Agentless head-to-head are all additionally evaluated there (scored under the canonical sympy runner, Section 5.2), so the load-bearing claims carry a held-out replication rather than a single-split calibration. The cross-tier frontier cells on Haiku 4.5 and Opus 4.7 (Section 8) are at $n=150$; the earlier cross-tier probes (budget signaling, the cheaper-model cascade) use the first 30 instances per cell. Cross-vendor cells on OpenAI `gpt-5-codex` use the first 150 instances for two-pass critique (matched to the Sonnet headline scale) and the first 30 instances for the variant probes (signaled budget and plan-then-execute). HumanEval is evaluated on the full 164-instance test set. MBPP is evaluated on the first 500 instances of the standard test split (task IDs 11–510).

We report observed pass rates and per-cell costs directly. A per-cell binomial standard error on pass rate is approximately ± 4 percentage points at $n=150$ and approximately ± 9 percentage points at $n=30$. Where a cross-vendor comparison rests on the smaller-sample evidence ($n=30$ on codex against $n=150$ on Sonnet for two of the three variant cells in Section 10.3), we flag the sample-size asymmetry in the body and in Section 15.

5.5 Sampling protocol

Each cell uses the vendor’s default sampling configuration: no explicit temperature override, no explicit random seed. The Anthropic Claude CLI [3] uses Anthropic’s default temperature; OpenAI `gpt-5-codex` is invoked via the Responses API [28] with its default reasoning configuration; Google Gemini 2.5-flash uses Google’s default sampling. The default-sampling choice matches how a deployer would invoke each vendor without per-cell tuning; it also means each cell carries an intrinsic run-to-run variance independent of sample size, quantified for the codex two-pass cell at Section 10.4 (66.7 ± 2.4 resolved instances over three independent runs at $n=150$).

5.6 Cost accounting

The framework computes per-attempt cost from the accumulated per-turn invocation records. Each call’s cost is the sum of three terms: fresh input tokens at the vendor’s published input rate,

cached input tokens at the vendor’s cached input rate, and output tokens at the vendor’s output rate. Cached input tokens are reported by the vendor in the API response, and the framework’s three shipped cost models read these fields directly from each vendor’s usage payload. The cache-discount factor is ten percent of the fresh input rate across all three vendors (Anthropic [2], OpenAI [27], and Google Gemini [12]).¹ Cached input tokens are billed at one tenth of the corresponding fresh input rate on each vendor’s current public pricing as of the pricing date stamped on the shipped cost models (2026-05-27).

Vendor prompt caching is a first-order effect on agent-loop billing. The shared per-turn agent loop re-sends the growing conversation history on every turn, and after the first turn typically eighty to ninety-five percent of the input is a cache hit. Naive cost accounting that multiplies total token counts by the published input rate can overstate real billed cost on agent workloads by roughly 2–7 $\times$, depending on the cache hit rate and the vendor. Every per-cell cost figure in this paper uses the cache-aware accounting described above.

Two cost components are not modeled in the shipped cost models and may differ in production deployments: batch-API discounts (approximately fifty percent off output tokens for asynchronous batched requests on Anthropic and OpenAI) and enterprise or committed-use pricing tiers. Users with either should subclass the shipped cost models. The shipped pricing tables need to be re-stamped when vendor pricing changes; the framework treats CostModel as a Protocol so the substitution is a single line in policy.yaml.

6 Single-knob mechanisms

Code-Capsules’ shipped variants compose four single-knob mechanisms: the turn budget the agent is given, whether that budget is signaled in the system prompt, the routing strategy that selects a variant per workload class, and (in some variants) a mid-session adaptation trigger. We measure each in isolation here; the composed variants are evaluated in Section 7.

6.1 Turn budget

Turn budget is the single most consequential knob on SWE-bench Lite. On Anthropic Sonnet 4.6 at the first thirty instances, the implicit-budget Pareto sweep (Table 3) is smooth and monotonic: 14/30 resolved (47%) at a five-turn cap, 22/30 (73%) at ten turns, 24/30 (80%) at twenty turns, and 27/30 (90%) at forty turns. Beyond forty turns the pass rate plateaus at the model’s natural termination depth; an unbounded one-hundred-turn cap matches within the same 90% band.

The cost-quality curve is concave then flat: each additional turn of budget buys progressively less pass-rate gain. No single budget dominates the others on cost and quality, so the budget knob alone is a continuous tradeoff rather than a Pareto-positive mechanism.

¹The ten-percent factor is the per-token cache-read rate. Gemini additionally bills a separate cache-storage fee per token-hour (roughly \$1 per million-token-hour on Flash, \$4.50 on Pro); for the short-lived caches in a single agent session this storage term is negligible relative to the read discount and is not modeled here.

Table 3: Turn-budget cost-quality sweep on Anthropic Sonnet 4.6, SWE-bench Lite first 30 instances. “Cap” column reports the fraction of sessions that hit the budget. All cells use the cache-aware accounting of Section 5.6.

Cell	Pass rate	\$/attempt	Cap
Implicit $b=5$	14/30 (47%)	0.185	83%
Implicit $b=10$	22/30 (73%)	0.287	63%
Implicit $b=20$	24/30 (80%)	0.430	27%
Implicit $b=40$	27/30 (90%)	0.503	3%
Unbounded $b=100$	27/30 (90%)	0.591	0%

6.2 Budget signaling

Whether the model is told its turn budget in the system prompt is a smaller and budget-dependent knob. Telling Sonnet “you have N turns” raises pass rate by 6.7 percentage points at $N=5$ (16/30 versus 14/30 implicit), is approximately neutral at $N=10$ on the first-thirty sample (19/30 versus 22/30 implicit, within binomial noise at this sample size), and provides no quality lift at that budget or larger. As a standalone knob, signaling is not Pareto-positive on Sonnet at any budget we evaluated. It becomes load-bearing when composed with other mechanisms (Sections 7.2 and 7.1) and shows a different pattern across model tiers (Section 8.6).

6.3 Routing

The framework’s routing strategy selects among shipped variants based on a per-task workload classifier. On monomorphic workload distributions, where every task in the input batch belongs to the same class, the router degenerates to a single-variant baseline because there is no diversity for it to discriminate. SWE-bench Lite is monomorphic in this sense (every instance is a real-world bug-fix task), and the router as a standalone mechanism is informative only on mixed-workload inputs, which our headline evaluation does not exercise at scale. Routing as a primitive remains useful for deployments that mix multiple workload classes through a single runner.

6.4 Mid-session adaptation

The framework supports two mid-session adaptation triggers as shipped primitives. A signal-driven turn-budget extension fires when in-session signals indicate the agent is stuck (file thrashing, repeated test failures, or specific traceback patterns), and extends the agent’s cap from an initial ten turns up to twenty-five. A simpler fixed two-pass escalation (Section 7.1) omits the signals and re-runs the agent unconditionally on a quality-gate failure. The signal-driven form did not match the fixed form on our SWE-bench Lite headline evaluations (Section 13.4); the fixed form is the one carried into the variant catalog and the external head-to-head (Section 11).

7 Adaptive execution variants

The framework ships seven adaptive execution variants that compose the single-knob mechanisms of Section 6 with pre-session or mid-session structure. This section describes the four variants with load-bearing roles at the first 150 SWE-bench Lite instances

Table 4: Variant cost-quality positions on Anthropic Sonnet 4.6, SWE-bench Lite first 150 instances. Two implicit-budget cells are included as baseline anchors. Pass-rate cells carry an approximate binomial standard error of ± 4 percentage points at $n=150$ (Section 5.4); per-row deltas under one standard error are statistically indistinguishable. The two-pass critique cell is priced with its second pass running unconditionally (Section 5.3). The two-pass critique and implicit-40 cells (98/150 versus 104/150) are statistically indistinguishable on pass rate at matched per-attempt cost. The stuck-signal injection cell (which extends the turn budget when in-session stuck signals fire) is oracle-assisted (Section 15).

Variant	Pass rate	\$/attempt
Signaled budget ($b=10$)	84/150 (56%)	0.17
Relevance ranker ($b=10$)	80/150 (53%)	0.18
Phase-staged ($b=35$)	88/150 (59%)	0.29
Stuck-signal injection ($b=10 \times 3$)	91/150 (61%)	0.33
Per-tool-class hint ($b=20$)	92/150 (61%)	0.28
Plan-then-execute ($b=20$)	93/150 (62%)	0.29
Two-pass critique ($10 \rightarrow 25$)	98/150 (65%)	0.41
Implicit $b=40$ baseline	104/150 (69%)	0.41

(two-pass critique, plan-then-execute, the relevance ranker, and per-tool-class allocation), reports the positions of the other three shipped variants that are Pareto-dominated at that sample size, and tabulates per-variant cost-quality cells in Table 4. Cross-tier transfer is in Section 8; cross-vendor transfer is in Section 10.

7.1 Two-pass critique

The idea is simple. Let the agent try once, check whether it worked, and give it a single second chance if it did not. The first attempt runs for ten turns. On session completion, the framework’s quality gate evaluates the candidate patch. If the gate fails (the test suite does not pass), the variant rolls the conversation forward and prepends a brief “critique your prior attempt and try again” prompt. The second pass then continues for up to fifteen additional turns. The total budget is capped at twenty-five turns across both passes.

The variant runs its second pass unconditionally on a quality-gate failure. Two-pass critique resolves 98/150 (65%) at \$0.41 per attempt on SWE-bench Lite first 150 with Sonnet. The comparison cell at the same model with an implicit forty-turn budget resolves 104/150 (69%) at \$0.41 per attempt: at matched per-attempt cost the two cells are within the binomial noise band, so two-pass is not a distinct cost knee. Plan-then-execute resolves 93/150 at \$0.29 (Section 7.2), five fewer at lower cost.

Where two-pass critique earns its place is the external anchor. It is the configuration carried through the matched head-to-head against Agentless (Section 11), where over the full 300 instances it resolves more (172 versus 152, +20, McNemar $p=0.0055$) at 13.8% lower cost per resolve, and the comparison replicates on the held-out split, with both systems re-measured under the same accounting rather than extrapolated. It ships as the menu’s balanced entry (Table 2): cheaper per attempt than the unbounded Sonnet baseline

(\$0.41 versus \$0.47) at a lower resolve count, for deployers whose budget is per-attempt rather than per-resolve.

The mechanism shape is a single-iteration instance of the verbal-reinforcement refinement loop introduced by Reflexion [32] and developed further by Self-Refine [24] and CRITIC [14]. Two-pass critique differs from those works in three ways relevant to the cost-quality framing in this paper: (i) the gate that triggers the second pass is the executable SWE-bench test suite, not a self-generated unit test or model-as-judge signal; (ii) the re-attempt is capped at a single iteration with a fixed turn-budget extension rather than a learned trial count; (iii) per-cell cost is reported under cache-aware accounting (Section 5.6), which on the agent-loop framing of SWE-bench Lite changes the absolute economics by approximately a factor of two to seven relative to naive per-turn token-count cost. The mechanism shape is inherited; the honest cost-quality characterization on a real software-engineering benchmark, under the framework’s calibration discipline and cache-aware accounting, is the contribution.

7.2 Plan-then-execute

Plan-then-execute prepends a short “produce a plan before any tool call” instruction to the system prompt and gives the agent a twenty-turn budget. On SWE-bench Lite first 150 with Sonnet, plan-then-execute resolves 93/150 (62%) at \$0.29 per attempt, matching the corresponding twenty-turn implicit-budget cell’s resolve count (93/150) at marginally lower cost and landing five short of the two-pass critique cell (98/150) at thirty percent lower per-attempt cost.

The mechanism is a single-step zero-shot specialization of the plan-then-act scaffold introduced by Plan-and-Solve prompting [35] and of the reasoning-and-acting interleaving of ReAct [41]: emitting a plan before any tool action shapes the trajectory the agent takes. Our specialization fixes the plan to a single up-front pass (no replanning mid-session) and inherits its turn budget from the corresponding implicit cell. Agent Capsules [30] ships no planning directive of its own; its closest mechanism by form is the output-guidance prompt (“Be concise. Aim for 300–400 words.”). The analogy is structural, not functional: both are soft prompt-level framings inserted once at session start that reshape session-wide behavior without any change to orchestration code, even though one shapes the agent’s action trajectory and the other its output verbosity. Cross-vendor behavior of plan-then-execute is discussed in Section 10.4.

7.3 Issue-relevant file pre-selection (relevance ranker)

A pre-session mechanism. Before the agent runs, the framework’s relevance ranker scans the target repository, scores each file by BM25-style match [31] against the issue text plus a short content peek, and prepends the top- N most-relevant file names (and optional snippets) to the agent’s system prompt. The agent receives this list as a starting hint with an explicit escape hatch (“if these do not look right, search yourself”).

On Sonnet at the first 150 SWE-bench Lite instances, the relevance ranker stacked on a ten-turn signaled budget resolves 80/150 (53%) at \$0.18 per attempt, four short of the ten-turn signaled budget

alone (84/150) at higher cost: the ranker block is cost-dominated on Sonnet. It remains useful as a wrapper on lower-tier models, where it lifts the weakest tier (Haiku, +8 instances; Section 8.2).

The mechanism has no Agent Capsules analog: research-task pipelines do not have a “find the file” sub-problem in the same sense. The closest Agent Capsules primitive is its cross-agent context injection suppression under group topologies, which operates on agent-dependency declarations rather than on source-file content relevance.

7.4 Per-tool-class allocation hint

A soft prompt-level mechanism: the system prompt encourages the agent to allocate turns roughly proportionally across tool classes (read, edit, test) rather than spending most of the budget in any one class. On Sonnet at first 150 the variant resolves 92/150 (61%) at \$0.28 per attempt, on the cost-quality frontier but dominated on quality by both two-pass critique and plan-then-execute. Useful as a calibration baseline and as a diagnostic for workloads where the agent over-allocates to a single tool class.

7.5 Other shipped variants

Two further shipped variants are not on the aggregate Pareto frontier at the larger sample size but carry class-conditional roles. The phase-staged budget allocation (a hard three-phase split with fixed turn caps per phase) over-constrains the agent on modern model tiers; on Sonnet at first 150 it resolves 88/150 (59%) at \$0.29, dominated by plan-then-execute, with no class-conditional cell where it wins.

The stuck-signal injection variant (which detects an in-session stall and injects corrective guidance mid-session) is more nuanced. On the first 150 instances it resolves 91/150 (61%) at \$0.33 and is dominated on aggregate. Conditioned on workload class, however, it is the strongest mechanism we measured on scientific-computing repositories. On the held-out scikit-learn cells (stratified-validation subset, $n=23$) it lifts pass rate by +9 percentage points over the plan-then-execute baseline, and the same pattern appears on the smaller astropy cell on first-30 (+33 percentage points over the baseline, $n=6$). The mechanism fires only on roughly 18% of instances overall (the signal-trigger condition is rare), but when it does fire on scientific workloads the per-instance lift is large. Two caveats scope this variant. Its cells are oracle-assisted (Section 15), so any deployment must substitute a deployable stop signal such as the execution verifier. The shipped deployment defaults accordingly list stuck-signal injection only as a scientific-workload override candidate rather than as a general-purpose default.

8 Cross-tier Pareto frontier

The same shipped variant catalog evaluated across the Anthropic 4.X family produces three differently-shaped cost-quality frontiers (Table 5, Figure 3). All three tiers are evaluated at $n=150$ on SWE-bench Lite. We report the three frontiers, the mechanism that transfers most clearly across tiers, and one mechanism finding that is tier-conditional.

8.1 Per-tier frontier shapes

The three frontiers differ qualitatively; Table 5 gives the per-cell pass rates and costs. The cross-tier sweep varies the turn budget, the dominant cost-quality lever, at a fixed prompting configuration, plus the relevance ranker, on all three tiers; this isolates the per-tier budget-scaling shape. The remaining adaptive variants of Section 7 are calibrated on Sonnet, and their cross-tier transfer is reported where measured (the ranker in Section 8.2).

Haiku climbs, then flattens. Its quality rises steeply through the implicit-40 cell (58.0%), then gains only six instances to the unbounded cap (62.0%), and stays well below Sonnet and Opus across the whole budget range.

Sonnet keeps paying. Its frontier does not level off inside the evaluated range: from the ten-turn signaled cell (56.0%) the gain to the implicit-20 cell is +9 instances, to implicit-40 a further +11, and to the unbounded cap a further +7 (74.0%), each leg buying resolves at a comparable marginal cost per attempt. The mid tier rewards added budget across the full sweep rather than saturating early.

Opus pays most up front. It jumps +19 instances from the ten-turn signaled cell (72.7%) to the twenty-turn implicit cell (85.3%, the cost-axis knee), after which returns diminish to +6 and +4 instances over the last two legs, reaching 92.0% at the unbounded floor.

8.2 The relevance ranker on lower-tier models

The relevance ranker stacked on a ten-turn signaled budget produces a tier-dependent effect over the bare signaled-budget cell, and the sign of that effect flips with model capability. On Haiku at $n=150$ the ranker raises pass rate from 17/150 (11.3%) to 25/150 (16.7%): an 8-instance, 5.3-percentage-point lift on the weakest tier. On Sonnet at $n=150$ it costs 4 instances (80/150 versus 84/150) and on Opus it costs 3 (106/150 versus 109/150), in both cases adding cost (\$0.317 versus \$0.311 on Opus) for a lower pass rate, so the bare signaled cell Pareto-dominates the ranker on both stronger tiers. The mechanism stays a recommended wrapper on the lower tier, where it adds resolves from a low base, but is dominated on Sonnet and Opus and is not a recommended default there (Section 8.5).

8.3 Opus at $n=150$: the strong tier holds its ceiling

The earlier first-30 Opus probe (27/30 at the twenty-turn implicit cell, 28/30 at the unbounded floor) projected a strong tier well clear of Sonnet. The matched- n Opus evaluation at $n=150$ confirms it: Opus reaches 92.0% (138/150) at the unbounded floor and 85.3% (128/150) at the twenty-turn implicit knee, against Sonnet’s 74.0% floor, an 18-percentage-point cross-tier gap at matched configuration. These are oracle-assisted absolute rates (Section 15); the load-bearing quantity is the relative +18-point gap, not the absolute level. The strong tier holds its ceiling across the broader 30–149 range rather than regressing toward the mid tier. The single-cell Opus ceiling at $n=150$ is 138/150 (92.0%) at the unbounded floor; the multi-cell oracle across the five Opus cells (any one resolves) is 143/150 (95.3%); and the absolute cross-vendor oracle across all shipped-variant configurations on four model families is 148/150 (98.7%) (Section 12).

Table 5: Cross-tier cost-quality cells on the Anthropic 4.X family, SWE-bench Lite first 150 instances. Each cell reports pass rate and per-attempt cost in dollars (cache-aware). All three tiers are at $n=150$; binomial standard error is approximately ± 4 percentage points per cell (Section 5.4). The Opus implicit and unbounded cells use a 900-second per-instance harness timeout (one instance hits it on the unbounded cell and is counted unresolved). Bold cells identify the cost-axis knee on each model tier. Absolute rates carry the target-test-identifier disclosure of Section 15, a symmetric upward bias across all cells; the load-bearing claims are the relative cross-tier deltas.

Cell	Haiku 4.5 ($n=150$)	Sonnet 4.6 ($n=150$)	Opus 4.7 ($n=150$)
Signaled $b=10$	17/150 (11.3%) @ \$0.074	84/150 (56.0%) @ \$0.174	109/150 (72.7%) @ \$0.311
Relevance ranker stacked	25/150 (16.7%) @ \$0.079	80/150 (53.3%) @ \$0.183	106/150 (70.7%) @ \$0.317
Implicit $b=20$	47/150 (31.3%) @ \$0.149	93/150 (62.0%) @ \$0.297	128/150 (85.3%) @ \$0.480
Implicit $b=40$	87/150 (58.0%) @ \$0.292	104/150 (69.3%) @ \$0.409	134/150 (89.3%) @ \$0.583
Unbounded ($b=100$)	93/150 (62.0%) @ \$0.449	111/150 (74.0%) @ \$0.466	138/150 (92.0%) @ \$0.604

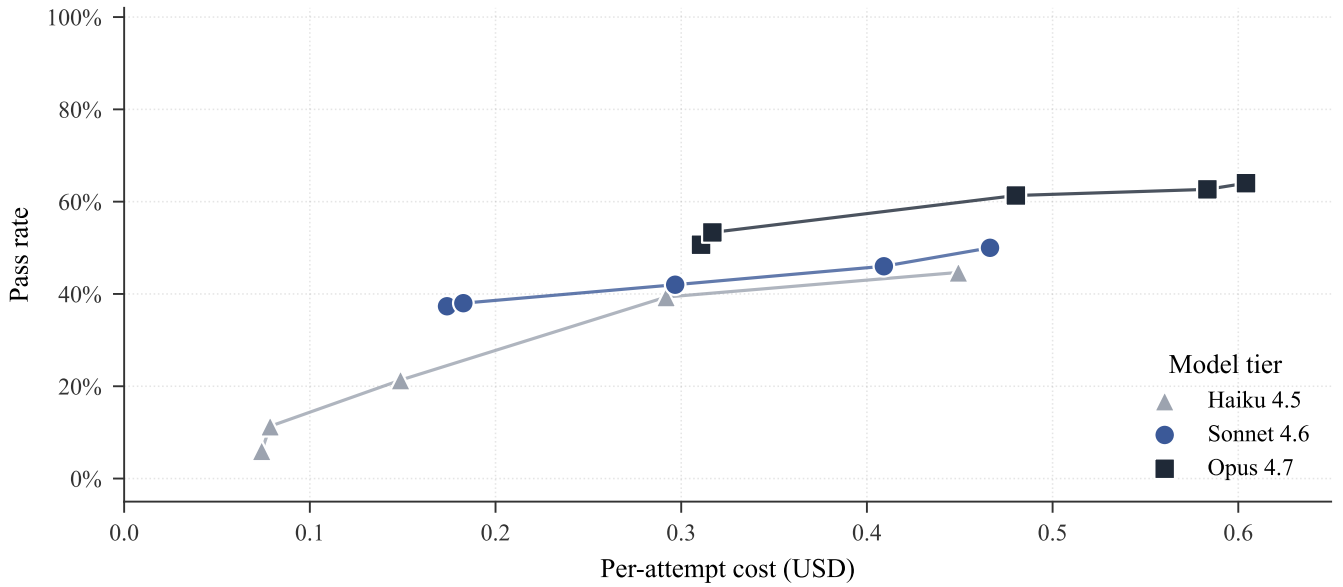


Figure 3: Cost-quality frontiers across the Anthropic 4.X family on SWE-bench Lite first 150 instances. Each marker is one shipped configuration evaluated on one tier; tier is encoded by marker shape and color (Haiku triangles, Sonnet circles, Opus squares). The line of each color is that tier’s upper-left Pareto frontier across the five cells reported here (signaled budget at ten turns, the relevance ranker, implicit budgets at twenty and forty turns, and the unbounded 100-turn floor). The three frontier shapes differ in where they bend. Haiku climbs steeply through the implicit-40 cell (58.0% at \$0.292) and then flattens, gaining six instances to the unbounded cap (62.0%). Sonnet keeps paying across the range, from 56.0% at the ten-turn signaled cell to 74.0% at the unbounded cap. Opus rises to 92.0% on the unbounded cell, with the cost-axis knee at the twenty-turn implicit cell (85.3% at \$0.480). The relevance ranker, stacked on the ten-turn signaled budget, helps only the weakest tier (Haiku, +8 instances) and is cost-dominated by the bare signaled cell on Sonnet and Opus. Haiku cells cluster on the cheap left of the cost axis while Sonnet and Opus operate in a higher cost regime, and that cross-tier cost-regime gap is itself a deployment signal. All three tiers are at $n=150$. Per-cell numerical values are in Table 5. Absolute rates carry the target-test-identifier disclosure of Section 15, a symmetric upward bias across all cells; the load-bearing quantities are the relative cross-tier deltas, not the absolute levels.

8.4 The strong tier as the quality-max menu entry

The cross-tier cells supply the menu’s quality leg (Table 2). The Opus twenty-turn implicit cell resolves 128/150 at \$0.480 per attempt, which is \$0.56 per resolve: more resolves than the unbounded

Sonnet baseline (111/150) at *lower* cost per resolve (\$0.56 versus \$0.63) and at comparable cost per attempt. A deployer buying quality should therefore step up a model tier at a bounded budget rather than buy unbounded turns at the mid tier; the unbounded Opus cell (138/150 at \$0.66 per resolve) buys ten further resolves at a

premium. The cheapest route into this quality band is a cascade rather than any single cell: running the cheap ten-turn signaled Sonnet cell and escalating its failures to the Opus twenty-turn cell resolves 132/150 at \$0.52 per resolve, Pareto-dominating the single Opus cell on both axes (Section 13.1).

8.5 Ranker effectiveness inversely scales with capability

The relevance-ranker mechanism does not simply stack on every tier. Its effect inverts as model capability rises: it composes positively with plan-then-execute on Haiku (+4 percentage points on the held-out evaluation), is approximately neutral on Sonnet at $n=150$ (Section 8.2), and regresses on Opus. Specifically, stacking the ranker on plan-then-execute on Opus drops pass rate by -6 percentage points; stacking it on two-pass critique on Opus drops pass rate by approximately -17 percentage points. The pattern is mechanistically interpretable: a more-capable model self-locates the relevant files in its first few turns at very low cost, so prepending a pre-computed file list is wasted tokens at best and a misdirection when the ranker’s BM25 surface guesses wrong. The shipped deployment defaults (Section 16) encode this asymmetry as an explicit rule: the relevance ranker stacks on Haiku, is optional on Sonnet, and should not be composed with any Opus variant.

8.6 Capability-headroom-mediated budget signaling

Whether telling the model its turn budget helps is conditional on model tier and budget. At a five-turn budget, signaling lifts Opus by 13 percentage points (19/30 versus 15/30 implicit), lifts Sonnet by 6.7 (16/30 versus 14/30), and does not lift Haiku (1/30 either way; Haiku at five turns is capability-bound regardless of signaling). The cross-tier pattern is consistent with a hypothesis we frame but do not claim to confirm: on coding tasks under a turn-budget framing, the signaling response is gated by the model’s spare reasoning capacity at the chosen budget rather than by its baseline capability.

This contrasts with Agent Capsules [30]’s budgeted-hint result on research-task pipelines, where smaller models responded more to budget cues than larger ones. The hypothesis we offer is consistent with both findings: research tasks operate the budget hint as a verbosity-control lever (smaller models gain more from verbosity reduction), while coding tasks operate the budget hint as a turn-pacing lever (only models with spare reasoning headroom at the budget benefit from being told it exists). The cross-tier evidence here is six data points (three tiers times two budgets at the comparison), and the hypothesis should be treated as directional rather than as a confirmed law.

8.7 HumanEval and MBPP across tiers

On HumanEval at fifty instances per tier, all three Anthropic models achieve 98–100% pass rate. On MBPP at fifty instances the range is 94–100%. The cost-quality frontier on these saturated benchmarks collapses to a single recommendation: Haiku at \$0.028 per attempt is the cost-axis choice for either benchmark, because all three tiers reach the floor and Haiku is the cheapest. The per-tier Pareto differentiation that exists on SWE-bench Lite is absent here;

the cross-tier framing in this paper is anchored on SWE-bench Lite for that reason.

9 Agreement on failure: a stopping signal and its honest deployment test

The cross-tier cells above buy coverage by spending more compute. A deployment also needs the opposite lever: a signal that says *stop spending more at this tier* on an instance no configuration at the current tier will resolve. We report one such signal: when two *diverse* configurations run at one model tier both fail, the instance is hard *at that tier*. This inverts the self-consistency heuristic [37]: where self-consistency reads agreement among samples as evidence of a correct answer, we read agreement among diverse configurations on *failure* as evidence that an instance is unsolvable at the current tier. Reading cross-model (dis)agreement as a label-free correctness signal is also explored in concurrent work [13, 15]; what we add is the executable, leak-free realization on SWE-bench Lite with a cost model and an anti-circularity check against a fully independent model-plus-harness reference (§9.3), together with the negative deployment result that separates the signal’s diagnostic value from its limited single-tier value. In one line: the signal is a sharp *diagnostic* of which instances are stuck at the current tier, but a poor *single-tier controller*. The flagged set is mostly recoverable at a stronger tier, so acting on the signal by quitting early throws resolvable work away; it earns its keep only as a trigger for escalating to a stronger model.

This section reports the signal in two registers, and is explicit about which is which throughout.

Vocabulary. Five terms recur below. *Diagnostic register*: a measurement that reads success or failure from the benchmark’s hidden evaluation verdict (the answer key); it bounds what the signal *could* achieve but is not deployable. *Deployment register*: the same question answered with a signal a live system actually has, such as an execution check or the project’s own test suite. *Oracle replay*: a diagnostic-register simulation that stops or escalates using the gold verdict; it yields an upper bound, never a shippable policy. *Gate-positive* / *gate-negative*: an instance the regression-suite check still flags as plausibly fixable, or not. *regok* (“regression-suite OK”): the deployable condition that the repository’s own existing tests stay green on the candidate patch; it is the live signal that replaces the gold verdict in the deployment register.

The *diagnostic* register asks what the signal measures. It reads failure from the benchmark’s evaluation verdict, so it is not deployable, but it establishes the signal’s precision and meaning: the signal itself (§9.1), oracle stopping replays (§9.2), an anti-circularity check against a fully independent model-plus-harness reference (§9.3), cross-tier replication (§9.4), and whether doom can be predicted before spending (§9.5).

The *deployment* register asks what survives when the gold verdict is replaced by a signal a deployment actually has. Its arc is one negative followed by two recoveries. The single-tier realization of the signal, integration-tested leak-free on all 300 instances (Section 5.3), *fails* its test and is retired (§9.2; in full in Section 13.5). A regression-suite gate then recovers most, not all, of what that realization destroyed (§9.6). Finally, the signal earns its keep as

an escalation trigger at the tier boundary (§9.8), not as a single-tier stopping rule: the flagged instances the single-tier rule would abandon are mostly recoverable one tier up.

9.1 The diverse-sample agreement signal

Everything in this subsection is a gold-graded diagnostic: failure is read from the benchmark’s evaluation verdict, which a deployment does not have. On Sonnet, across the $n=126$ instances every shipped configuration attempts, 20 are doomed at the ten-turn tier: no configuration drawn from a budget of ten attempts resolves them. The oracle union across those attempts (any one resolves) reaches 106/126, so the gap between a single attempt and the full oracle union is a wide addressable middle, and the doomed set is the thin residue below it. A doomed-instance detector must fire on that residue without burning resolvable instances.

Requiring *two diverse* configurations to both fail is a sharper detector than reading one. A single configuration’s failure is too blunt: the baseline single-pass configuration failing predicts tier-local doom at 64.5% precision, firing on 31 instances, 11 of which a different configuration would have resolved. The two-configuration signal lifts this to 90.9% precision. It fires on 22 instances with two false terminations. The lever is the *agreement of diverse configurations*, not the failure of any one.

The 90.9% figure is an in-sample best case, not the number a deployer should plan around. It is the best of all 45 pairs of shipped configurations at this tier, scored in-sample. This pair ranks first and is the only pair above 90% precision; the median pair sits at 66.7% (Figure 4, in-family bars). The deployment-relevant precision is the independent figure of Section 9.3, which drops to roughly 22%: measured fully out-of-family, most of the flagged set is recoverable at a stronger tier, and only a small residue is unresolved everywhere. The precision gap between the median pair and the best pair is large enough that the partner choice is itself a tuned parameter rather than an incidental one. One member of the best pair is the signal-injection variant, which is oracle-assisted (Section 15); unions and ceilings that include it inherit that label.

The same two diverse configurations also supply a coverage lever in this gold-graded replay: run both and keep whichever clears the quality gate. This two-configuration select resolves 104/126 at a modeled \$85.02, against 95/126 at \$43.20 for the single baseline configuration, all but two of the 106 oracle-resolvable instances, at roughly 2× the baseline cost. Adding a third configuration resolves no further instance (104/126): the agreement signal correctly reports that the residue is unreachable at this tier, so the third attempt is wasted spend. Whether the pair’s two levers (select the resolver, stop on agreed failure) survive replacing the gold grade with a deployable signal is the integration test of Section 9.2; they do not.

9.2 From oracle replay to the deployable integration test

This subsection first measures the signal against an upper bound it cannot ship, then makes it deployable. The bound is an *oracle replay*: a stopping rule that reads the gold verdict, so it is diagnostic only. The deployable version replaces that verdict with the regok check, the project’s own tests staying green.

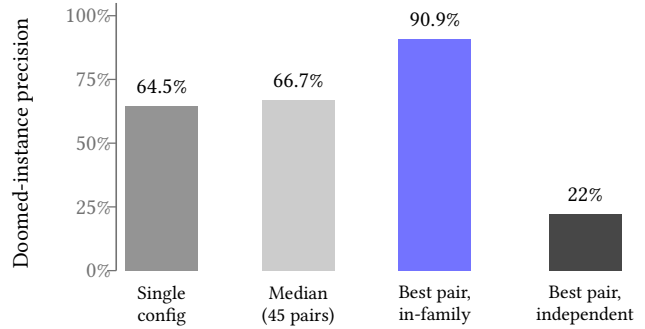


Figure 4: Doomed-instance precision of the diverse-sample agreement signal on Sonnet, SWE-bench Lite first 150 instances. The three left bars measure precision against other same-tier configurations (in-family): a single configuration’s failure (64.5%), the median of all 45 configuration pairs (66.7%), and the best diverse pair (90.9%). The right bar measures the same best diverse pair against an *independent* reference that varies both the model tier and the agent harness (22%); the gap from the in-family bar to the independent bar is the share of flagged instances a stronger tier recovers. That gap is large: most of what the in-family signal flags as doomed is recoverable one tier up, so the signal measures tier-relative difficulty rather than absolute task hardness. This is why its deployable use is an escalation trigger, not an early-abandon rule. All four bars are gold-graded diagnostics: failure is read from the benchmark’s evaluation verdict. Deployed with a non-gold verifier signal, the same architecture’s ship precision is 75–77% (§9.2).

The agreement signal is a stopping rule, so one informative analysis is its distance from the *oracle* stopping policy: a controller that, after the same two-configuration probe, escalates only the instances some configuration would resolve and abandons the rest. In the gold-graded replay (failure read from the evaluation verdict), the best diverse pair flags 22 both-fail instances at a common probe cost of \$85.02. Escalating every flagged instance to a third (plan-then-execute) configuration recovers none of them. The oracle escalates only the two flagged instances another shipped configuration resolves, reaching 106/126. The agreement-based policy abandons all 22, reaching 104/126: within two instances of the omniscient oracle at the same probe cost. A held-out replay (second 150 instances) shows the same shape. These replays describe what the signal is worth *when failure is read perfectly*; they are analyses, not deployment results, because the failure reads consult the held-out verdict.

A real controller cannot read the benchmark verdict; it must judge failure from its own evidence. The deployment question is therefore what survives when the gold grade is replaced by a signal a deployment actually has. We integration-tested exactly that on all 300 instances, with every control decision driven by deployable signals and the benchmark verdict used only for scoring (Section 5.3). The controller has three parts:

- a diverse pair, the unbounded baseline plus the signal-injection partner;
- a verifier that selects between the two candidates using a generated fail-on-base reproduction script (test-based selection in the lineage of CodeT and AlphaCode [7, 22]; see §14.4);
- the agreement rule, which abandons instances whose candidates both fail that verifier.

The verifier-driven architecture ships 100 of 150 first-half instances and 105 of 150 held-out, with 28 false-positive ships on the first half (ship precision 75%), and wrongly abandons 28 instances on the first half and 19 held-out that carried a resolving candidate. Deployed honestly (an abandoned instance ships nothing), it resolves 84 versus the unbounded baseline’s 111 on the first half and 76 versus 93 held-out. The governor’s deployed precision is bounded by the verifier signal, not by the agreement mechanism: every wrong abandon traces to a reproduction false-negative, where the generated script fails a candidate the held-out tests would pass. The single-tier realization is therefore retired as a deployment default; Section 13.5 reports the full negative, and Section 9.6 reports the gate that recovers most of the loss.

9.3 Anti-circularity: the flag is real but tier-relative

The 90.9% figure is measured against other same-tier configurations, so on its own it could be in-family correlation: two configurations that share a model might fail together because they share the model’s blind spots, not because the task is hard. The load-bearing test is whether the flagged-doomed set survives a genuinely independent reference. We run two controlled contrasts on the 27 flagged instances.

Varying the agent harness only (a different, independently capable agent harness running the same model) recovers 0 of the 27 flagged instances. That independent harness is not weak: it resolves 87/150 overall, comparable to the configurations that produced the signal, yet it resolves none of the instances the signal flags, so the flag is not one agent design’s blind spot. Varying the model instead (escalating the same harness to a stronger tier) recovers most of the flagged set, which shows that “doomed” is largely tier-relative: a stronger model breaks the majority of these instances. Combining both axes (vary the model *and* the harness) gives an independent precision of roughly 22% (6 of 27 flagged instances remain unresolved under the independent model-plus-harness reference; Figure 4, right bar). The signal loses most of its precision when measured fully out-of-family, and what it loses is exactly the tier-relative tail a stronger model recovers: the lever Section 13.1 prices and the escalation trigger of Section 9.8 acts on. The small residue that survives every configuration, model, and harness is a handful of instances, so the agreement signal at one tier is a reliable trigger to escalate, not a verdict that an instance is hopeless.

9.4 Cross-tier replication and the tier-specific partner

The mechanism replicates across all three model tiers. On the cheap tier the best diverse pair predicts doom at 95.9% precision (fires on 49 instances, 2 false terminations); on the mid tier it is the 90.9%

pair above; on the strong tier the agreement signal fires as well, since its two strong-tier configurations are diverse enough that their oracle union (143) exceeds the best single configuration (138). Agreement of diverse configurations is the transferable mechanism, not any single configuration’s behavior.

The honest caveat is that the *best partner* is tier-specific. The signal-injection variant (Section 8.6) is a strong partner at the mid and strong tiers but collapses on the weakest model, where it cannot produce reasonable stage-one patches to disagree about; there a budget-lever configuration is the better partner. A cross-model deployment selects the diverse pair per tier rather than fixing one universal pair, the same tier-conditional pattern the relevance ranker exhibits (Section 8.5). The signal itself (two diverse configurations agreeing on failure predicts doom at this tier) transfers; the configurations that instantiate it do not.

9.5 Pre-probe prediction: doom is hard to forecast before spending

The governor reads the *outcome* of a cheap probe. A natural question is whether the probe is necessary at all: can an instance be flagged as doomed from features known before any compute is spent? We train a shallow gradient-boosted decision-tree classifier (not a language model) to predict doom-at-tier from turn-zero features (repository, problem-statement length, model tier), pooling the three tiers into 330 (instance, tier) rows and evaluating with group-aware cross-validation that keeps every instance’s tier-rows within a single fold.

The signal is deliberately modest. Turn-zero features predict doom at an area under the ROC curve of roughly 0.56, above a repository-only floor of 0.54 but with a wide band: problem-statement length and model tier carry most of the lift, and repository identity alone is close to a coin flip. Adding the early dynamics of the ten-turn probe (its tool-call mix, files touched, whether it ran the test suite, traceback counts) does not separate any better (AUC 0.58, inside the same wide band), a result that reproduces under both the gradient-boosted and a logistic model: at the ten-turn horizon the behavioral trace is too low-variance to separate doomed from resolvable instances. The static prior transfers weakly between the two lower tiers (training on two tiers and testing the held-out third gives AUC 0.656 on Sonnet and 0.590 on Haiku); the strong tier is too rarely doomed to serve as a held-out test.

The contrast with the outcome-based signal is the point. Predicting doom *before* spending compute is only weakly possible from what the task looks like; detecting it *after* a single cheap probe round, by reading whether two diverse configurations agree on failure, is near-optimal under oracle grading (Section 9.2) and bounded by the verifier signal in deployment. Doom is a property the agent reveals by attempting an instance, not one legible in the issue text, which is why the operative stopping signals are outcome-based and a turn-zero predictor serves only as a weak, free pre-screen. A companion negative supports the same conclusion: pooling both splits’ abandon decisions (30 positive instances across 300), no learned model (logistic, shallow tree, or gradient-boosted) clears the pre-registered out-of-fold tests against the simple regression-suite rule of Section 9.6; the rule is hard to beat at this positive-class size, and we ship the rule.

9.6 A regression-suite gate restores the wrong abandons

The integration test’s honest loss (84 versus 111, 76 versus 93) is concentrated in one class: candidates whose generated reproduction script false-negatives. The repairs are real: on the held-out split, all 19 of the reproduction scripts that failed a gold-resolving candidate are themselves over-specified (even the benchmark’s reference patch fails them). The fix is therefore a second, independent check. The *regression-suite gate* ships a candidate when its reproduction script passes *or* the repository’s pre-existing regression suite stays green on the selected candidate; agreement-on-failure then routes the remainder to escalation or hand-back instead of silent abandonment.

The gate is gold-independent and cheap: the regression suite is already run as part of validation, so the rule adds no model calls. On the first 150 instances it lifts the honest controller from 84 to 99 at zero added cost, recovering most of the wrong-abandon class but staying short of the unbounded baseline’s 111; a regression-green candidate resolves with probability 0.792. On the held-out 150 it lifts 76 to 89 against the baseline’s 93: short by four instances, each a regression-gate false-negative. The gate strictly dominates the agreement-only controller on both splits (+15 and +13 at zero added cost), so it ships as the no-silent-abandon default, but we claim recovery, not parity. Two scope caveats. The rule was derived on 38 abandon rows, 68% of them django, and the per-candidate signal attenuates off django. A negative gate is not doom: most of the gate-negative set is recoverable at a stronger tier (Section 9.8), so any policy that declines gate-negative instances outright needs per-workload re-justification.

9.7 The routing ceiling: why a single-tier router cannot pay for itself

The integration test’s cost arithmetic generalizes. A router that picks one of two configurations per instance can, at best, add the instances that its second choice resolves but its first choice misses; that ceiling is what the union of the two solve-sets buys over the better single configuration alone. The most any per-instance selection over the diverse pair can ever harvest is the oracle union minus the best single member: +9 instances on the first 150 (104 versus 95), with a comparable margin held-out; and these figures are themselves optimistic, since the second member’s cells are oracle-assisted (Section 15). The pair costs roughly twice a single configuration, so a few instances of routing headroom cannot pay for a doubled bill; in the event, the deployed selection carried *zero* net resolves over simply preferring the baseline’s candidate, across all 300 instances.

Routing across *tiers* pays. An oracle that picks the best logged configuration per instance reaches 148/150 at \$36.04 total (\$0.243 per resolve), and the gap to a deployable policy is small. Running the cheap signaled-budget Sonnet cell and escalating its failures to the bounded twenty-turn Opus cell resolves 132/150 at \$69.05, which Pareto-dominates the best single configuration (the Opus twenty-turn cell, 128 at \$72.03) on both axes: more resolves at lower cost, and the win holds under every tie-break ordering of the escalation queue. Unconditional escalation is what pays: tier-one is cheap and few failures remain, so escalating every failure is affordable; the

regression-gated variant alone (121/150) does not reach the bar, because the gate is imperfect (Section 9.8). The cross-tier cascade is therefore a deployable product; it is per-instance routing *within* a tier, not across tiers, that cannot pay for itself.

9.8 The signal as an escalation gate between tiers

The same gate that is value-destroying as a single-tier abandon rule becomes value-adding when the stopped set has a destination. Escalating the flagged weak set to the strong tier instead of abandoning it reaches 85/150 on the first 150 at roughly \$1.58 per resolve: +21 over the honest single-tier controller’s 64. The strong tier recovers nearly the entire flagged set: 21 of the 21 gate-positive instances it receives *and* 15 of the 17 gate-negative instances it would otherwise also have been sent. The regression gate therefore does not separate recoverable from doomed at this tier boundary, because almost every flagged instance is recoverable one tier up; what pays is escalating the whole flagged set, not gating it. Stopping, in other words, pays at the *tier boundary* (spend more model on the flagged set, hand back only the small residue), not inside one tier. One caveat carries over: the strong-tier lift is a single run.

The regression-suite split still ranks which failures to escalate first. It partitions the cheap-tier patch-bearing failures into a *close* set (17: regression green, reproduction failed, nearly there but out of budget) and a *lost* set (32: regression red, on the wrong track). The close set was the cleaner escalation candidate, but under canonical scoring the strong tier recovers most of both, so the gradient that once justified handing back the lost set has largely flattened: the deployable rule is to escalate the flagged set broadly and use the regression-suite split only to order escalation under a constrained budget, the same signal that restores the wrong abandons in Section 9.6.

10 Cross-vendor extension via ModelClient

The framework’s eighth extension primitive, `ModelClient`, is a vendor-agnostic agent-runtime abstraction. We ship four implementations, evaluate the shipped variant catalog cross-vendor on HumanEval, MBPP, and SWE-bench Lite, and report two findings about which mechanisms transfer across vendor families.

10.1 The ModelClient primitive

Each `ModelClient` implementation adapts the framework’s per-turn invocation records to a specific vendor’s tool-calling API and reads back the vendor’s usage telemetry into the framework’s invocation-result schema. We ship four implementations: the Claude Code CLI [3] for Anthropic, the Chat Completions endpoint for OpenAI’s general-purpose models, the Responses API [28] for OpenAI’s reasoning-mode models (including gpt-5-codex, which the Chat Completions endpoint declines to serve), and the Google Gemini API for Gemini [12]. Each implementation registers itself with the framework’s primitive registry on package import if and only if the vendor SDK is importable at runtime; the framework stays lightweight for users who do not need cross-vendor.

Table 6: Cross-vendor pass rate and per-instance cost on HumanEval ($n=164$) and MBPP ($n=500$). Same shipped variant and quality gate across vendors; only the ModelClient primitive differs. Pass rates and per-instance costs reported under cache-aware accounting.

Benchmark	Vendor	Pass rate	\$/instance
HumanEval	gpt-5-codex	161/164 (98.2%)	0.0161
HumanEval	Gemini 2.5-flash	152/164 (92.7%)	0.0033
MBPP	gpt-5-codex	265/500 (53.0%)	0.0219
MBPP	Gemini 2.5-flash	251/500 (50.2%)	0.0014

10.2 Cross-vendor parity on HumanEval and MBPP

We ran HumanEval (full 164-instance test set) and MBPP (first 500 instances of the standard test split) cross-vendor with the framework’s variant set held constant. The new cells are OpenAI gpt-5-codex via the Responses API and Google Gemini 2.5-flash; Anthropic Sonnet 4.6 numbers are reference cells from the per-tier evaluation of Section 8.7.

The cost ratios are $4.8\times$ on HumanEval and $15.9\times$ on MBPP, both in Gemini’s favor (computed on the unrounded per-task costs; the table rounds to four decimal places). On these saturated single-completion benchmarks the framework’s mechanisms do not differentiate the vendors meaningfully because the agent-loop framing itself is the binding constraint: MBPP’s natural form is single-shot completion (under which Claude is reported to score around 98% in the literature), and an agent loop wrapping it hits a framing ceiling that handicaps both vendors equally. We use these benchmarks as cross-vendor parity sanity checks; we use SWE-bench Lite for cross-vendor capability comparisons where agent behavior is required.

10.3 Cross-vendor variant ladder on SWE-bench Lite

We evaluated three shipped variants on gpt-5-codex against the matched Sonnet 4.6 cells from Section 7. All three codex cells are evaluated at $n=150$ to match the Sonnet sample size.

10.4 What transfers, what does not

Four findings emerge from the matched- $n=150$ comparison.

First, **the variant ranking transfers across vendors at $n=150$** . On Sonnet the ladder is monotonic: signaled $\rightarrow$ plan-then-execute $\rightarrow$ two-pass critique moves pass rate $56.0 \rightarrow 62.0 \rightarrow 65.3\%$. On codex the same ordering holds: $30.7 \rightarrow 32.7 \rightarrow 44.5\%$. The relative ordering of the three mechanisms is stable across two unrelated vendor families, which is evidence that it reflects the scaffolds rather than any one model’s idiosyncrasies.

Second, **the iteration-axis scaffold (two-pass critique) adds lift on both vendors, and more on codex**. Against plan-then-execute, the always-run second pass resolves 98 versus 93 on Sonnet (+5 instances, +3.3 pp) and a mean of 66.7 versus 49 on codex (+17.7 instances, +11.8 pp; the gain is robust across the codex variance band, +13 to +23). The larger codex gain is the honest

reading of the always-run configuration: a second full attempt recovers instances the first pass left unresolved. It does not, however, lift codex to Sonnet’s level: codex’s two-pass ceiling (44.5%) sits about 21 percentage points below Sonnet’s (65.3%), and the gap is dominated by codex’s empty-patch failure mode (third finding).

Third, **codex’s failure mode is the empty patch, and its run-to-run variance is large**. Three independent runs of the codex always-run two-pass cell under identical configuration resolved 66, 62, and 72 instances on the same 150 (mean 66.7 ± 2.4 standard error, range [62, 72]). Only 48 instances are resolved by all three runs; the three-run union is $84/150$ (56.0%), well above any single-run cell. Pairwise Jaccard similarity is 0.66–0.72. The variance is not infrastructure: zero API timeouts and zero harness errors in any run. The dominant failure mode is an empty patch (no `<patch>` block emitted) on 62–70 instances per run ($\sim 44\%$); the same instance flipping between an empty patch and a candidate patch on resampling accounts for most of the run-to-run delta. That points to an unstable output contract rather than a stable reasoning ceiling, and it caps the headroom of any iteration-based scaffold on codex: a second pass cannot critique a patch that was never emitted. A single codex cell should be read with a wider confidence interval than its denominator suggests.

Fourth, **vendor cost-efficiency is a cost-min versus quality-max choice, not a global ordering**. On HumanEval and MBPP (Section 10.2), Gemini 2.5-flash is the cost-axis winner at single-digit fractions of a cent per attempt. On SWE-bench Lite, codex occupies the cost-min corner: the codex signaled-budget cell at \$0.142 per attempt is the cheapest cell on the cross-vendor Pareto frontier but caps at 30.7%, while the cheapest Sonnet cell at \$0.174 delivers 56.0% for 22% more cost. At the matched always-run two-pass cell, codex is cheaper on *both* cost axes: \$0.193 per attempt versus Sonnet’s \$0.408, and \$0.43 per resolved instance ($\$0.193 \cdot 150/66.7$) versus Sonnet’s \$0.62 ($\$0.408 \cdot 150/98$). That per-resolve advantage is real but must be read with its cause: codex is cheap partly because it fails cheaply, emitting an empty patch on $\sim 44\%$ of instances rather than spending tokens on a candidate, which is also why its quality ceiling stays well below Sonnet’s. Codex is therefore the cost-min end of the cross-vendor frontier and Anthropic Sonnet and Opus the quality-max end, with the Sonnet cells anchoring the mid-cost region. Vendor choice is not a global ordering; it is a per-workload and per-cell choice that the framework’s measurement loop is designed to make explicit.

11 Head-to-head against Agentless

The cells above anchor against a binomial standard-error band rather than any single external number (Section 5.4). To place the framework directly against a published baseline under matched conditions, we re-ran Agentless [39] on the full 300-instance SWE-bench Lite set with Anthropic Sonnet 4.6, scoring every patch through the same Docker evaluation pipeline used for our own cells. Agentless patch selection uses oracle@ k (each candidate is run against the held-out tests and the first resolver is kept), a *generous* upper bound on its published rerank. This is the same oracle@ k leak flagged in Section 13.6, applied to the baseline rather than to our own cell, so the comparison handicaps the framework in the baseline’s favor and cannot flatter it.

Table 7: Variant cost-quality cells on SWE-bench Lite first 150 instances across Anthropic Sonnet 4.6 and OpenAI gpt-5-codex via the Responses API. All cost figures are cache-aware; binomial standard error is approximately ± 4 percentage points per cell (Section 5.4). The two-pass critique cells use the always-run (leak-free) second pass (Section 5.3) for both vendors, the same configuration as the head-to-head cell in Table 9; the second pass runs unconditionally (10+15 on Sonnet, 10+10 on codex), so the cost is higher than a gated second pass but the measurement carries no test-feedback leak. The codex two-pass figure is the mean of three independent runs (66, 62, 72 resolved; ± 2.4 standard error, range [62, 72]); its run-to-run variance is analyzed in Section 10.4.

Variant	Sonnet 4.6 ($n=150$)	gpt-5-codex ($n=150$)
signaled budget ($b=10$)	84/150 (56.0%) @ \$0.174	46/150 (30.7%) @ \$0.142
plan-then-execute ($b=20$)	93/150 (62.0%) @ \$0.287	49/150 (32.7%) @ \$0.297
two-pass critique	98/150 (65.3%) @ \$0.408	66.7/150 (44.5%) @ \$0.193

Table 8: Per-repository pass-rate breakdown for the always-run two-pass critique cell at $n=150$. The first 150 SWE-bench Lite instances span five repositories; django is by far the largest cell. Sonnet is the always-run cell from Table 9; the codex column is a single representative run (run 1 of three; the three-run mean is 66.7, Section 10.4).

Repository	n	Sonnet 4.6	gpt-5-codex
astropy	6	3/6 (50.0%)	3/6 (50.0%)
django	114	81/114 (71.1%)	55/114 (48.2%)
matplotlib	23	11/23 (47.8%)	7/23 (30.4%)
mwaskom	4	2/4 (50.0%)	1/4 (25.0%)
pallets	3	1/3 (33.3%)	0/3 (0%)
Total	150	98/150 (65.3%)	66/150 (44.0%)

Table 9: Head-to-head on SWE-bench Lite, Sonnet 4.6, matched Docker eval and modeled cost surface (Section 5.6). Agentless uses oracle@ k selection. The Code-Capsules cell runs the two-pass configuration whose second pass runs unconditionally (Section 5.3). Costs are the full-300 cells; per-split, the two-pass cell is \$0.408 per instance on the first 150 and \$0.463 on the held-out 150.

Harness	first-150	held-out	full-300	\$/inst	\$/res
Code-Capsules (two-pass)	98	74	172/300 (57.3%)	0.445	0.776
Agentless (oracle@ k)	87	65	152/300 (50.7%)	0.456	0.900

The two-pass critique cell resolves more, at lower cost per resolve, despite the oracle handicap: +20 instances (+6.7 percentage points) at 13.8% lower cost per resolve and 2.4% lower modeled cost per instance over the full 300. The quality edge is significant on the paired data: of the 48 instances the two harnesses disagree on, 34 favor Code-Capsules and 14 favor Agentless (exact McNemar two-sided $p = 0.0055$). The claim is therefore a both-axes win: the framework’s balanced configuration resolves more *and* costs less per resolve, on both splits (held-out: 74 versus 65 at \$0.94 versus \$1.07 per resolve). Per repository, Code-Capsules resolves at least as many instances as Agentless on ten of the twelve, with the margin concentrated on django (81 versus 74 of 114), sympy (37 versus 31 of 77), matplotlib (11 versus 8 of 23), scikit-learn (16 versus 14 of 23), and sphinx (8 versus 6 of 16). On the remaining

two repositories Agentless resolves one more instance than Code-Capsules: requests (0 versus 1 of 6) and xarray (2 versus 3 of 5). Two caveats bound the comparison. Each cell is a single run. Twenty-three Agentless rows produced no patch and hence no verdict (ten were sympy edit-localization crashes, the rest empty patches; run infrastructure rather than method failures); each was given a symmetric fresh recovery attempt and none produced a scoreable patch, so the resolve-count lead is not an artifact of those rows. And an evaluation-protocol asymmetry favors our side (Section 15): the framework’s solver prompts include the benchmark’s target-test identifiers (the names of the tests a fix must pass), which the issue-only Agentless pipeline never receives. This advantage is present in the reported numbers; a bounding re-run with the identifiers withheld is open work.

The unbounded Sonnet baseline anchors the held-out split more strongly: on the commonly-scored held-out instances it resolves 84 against Agentless’s 64 (+20, McNemar $p \approx 8 \times 10^{-4}$) at per-instance cost parity (\$0.466 modeled for the baseline against \$0.460 measured for Agentless; the baseline’s held-out cost is modeled from its calibration-split accounting, not joined from a held-out cost sweep, and is labeled accordingly).

Evaluation correctness (sympy). Both harnesses are scored under sympy’s canonical bin/test runner, validated by reference patches (Section 5.2): Sonnet 4.6 resolves 37/77 under the two-pass cell against 31/77 for Agentless oracle@ k . The workload-class implication is taken up in Section 12.

12 Workload reachability

Across every model and configuration we ran on SWE-bench Lite at $n=150$, how many of these instances can the portfolio resolve at all? Taking the union of resolved instances across all four vendors evaluated (Anthropic Haiku 4.5, Sonnet 4.6, Opus 4.7, and OpenAI gpt-5-codex) and every shipped variant on every tier, the union reaches 148/150 (98.7%). The complementary set is just 2 instances (1.3%), both matplotlib (matplotlib-22711 and matplotlib-25498); on the first 150 instances the portfolio leaves essentially no workload unreachable. These are oracle-assisted rates (the solver prompts carry the benchmark’s target-test identifiers, Section 15) taken as an oracle union over configurations, an upper bound rather than a single deployable policy.

Table 10: Floor-unresolved instances on Anthropic Sonnet 4.6 at $n=50$, first 50 SWE-bench Lite instances. Each cell produces a candidate patch that fails the post-fix test suite. The `astropy-14365` cell is on the capability-ceiling boundary (resolved by the implicit 40-turn baseline and by the relevance ranker variant; not resolved at the unbounded budget on its own).

Instance	Failure pattern	Turns
<code>astropy-14182</code>	Numeric / parsing edge case	12
<code>astropy-14365</code>	Numeric / parsing (boundary)	11
<code>django-11019</code>	Large multi-file fix scope	30
<code>django-13158</code>	Multi-test interaction	24
<code>django-13220</code>	Multi-test interaction	15

Opus alone covers 143 of the 148 reachable instances; the other five are resolved by non-Opus configurations (one `astropy` and four `django` cells), so the cross-vendor diversity benefit is a modest +5 over the strong tier rather than zero. With only two instances unreachable by anything we ran, there is no sizable universally-doomed core to abandon: the stopping signal of Section 9 earns its keep as a tier-boundary escalation trigger, not as an early-abandon rule.

The two unreachable instances are both `matplotlib`; `django`, the largest repository in the split, is fully covered. The failure mode on the residual pair is that every candidate patch fails the post-fix test suite, not that the agent emits no output. The same pattern appears in the harder Sonnet-floor-unresolved instances at the earlier first-50 probe (Table 10), which grouped that smaller sample into three patterns: numeric or parsing edge cases (two `astropy` instances), a single large multi-file fix whose scope exceeds the agent’s single-session reasoning capacity (`django-11019`), and multi-test interactions where the candidate patch is partially correct but does not cover every affected test case (`django-13158`, `django-13220`). Those Sonnet-floor misses are recovered by stronger configurations in the portfolio; only the two `matplotlib` instances resist every configuration.

The held-out portion of SWE-bench Lite (instances 150–299) is dominated by `sympy` (77 of 150). `sympy` is *not* a workload ceiling: Sonnet 4.6 resolves 37/77 (48%) under the two-pass cell (in the same range as its overall full-300 rate), and the Agentless baseline resolves 31/77 (Section 11), both scored under `sympy`’s canonical bin/test runner validated by reference patches (Section 5.2), so `sympy` is not a budget-insensitive class. The disclosure that ships in `policy.yaml` therefore scopes only the genuinely budget-insensitive instance tail below, not a symbolic-math class.

The residual `matplotlib` pair is workload-bound rather than budget-bound: spending more turns on these instances does not resolve them, and Table 10 illustrates the same failure mode on the harder Sonnet-floor misses. Mechanisms that might unlock such instances (ensembling, pass-at- k aggregation across independent sessions, multi-shot self-debug protocols) are out of scope for this paper and are noted in Section 16 as future work.

13 Negative results and dropped mechanisms

Several mechanisms evaluated alongside the shipped variant catalog sit off the Pareto frontier or are otherwise not recommended for deployment. We report them because the negative result is the load-bearing finding in each case; the largest (Section 13.5) retires what an earlier stage of this work treated as its central deployable result.

13.1 Cheaper-model cascade

A cheaper-model-first cascade (run the agent on Haiku 4.5 with a ten-turn signaled budget, then fall back to Sonnet 4.6 on a quality-gate failure) was evaluated as a potential cost-axis Pareto knee. Haiku at the matching cell resolves 2/30 (7%) of instances that Sonnet’s cell resolves 18/30 (60%) at first 30 instances. Crucially the Haiku pass set is a strict subset of Sonnet’s: Haiku resolves no instance that Sonnet does not at the same cell, so the cascade union is 18/30 (60%), +0 instances over Sonnet alone. The cascade cost is the Haiku attempt plus the Sonnet attempt on the Haiku failures: roughly +41.5% relative to running Sonnet alone, for zero additional resolved instances. The arithmetic is uneconomical even before accounting for the added latency of two sequential agent sessions.

The failure here is the *blind* cascade: it pays the stronger-model attempt on every cheaper-model failure, most of which are doomed regardless of tier. The diverse-sample agreement signal of §9 corrects exactly this. A *gated* cascade escalates only the instances the agreement signal flags as doomed, rather than every quality-gate failure, so the stronger-model spend is concentrated on the subset where tier could plausibly matter. The gate is the right primitive; the open question is whether the resolves it recovers are worth their cost.

They are recovered for the majority of the doomed set, at low multiples of the base cost per resolve. Against the two-configuration diverse-select base operating cost of roughly \$0.82 per resolve in the gold-graded replay (\$85.02/104 resolved, §9.1), escalating the agreement-doomed Sonnet set to Opus recovers 78–81% of them at \$1.37 per recovered resolve with one stronger configuration and \$2.71 with two. Escalating from the cheaper tier behaves the same: the agreement-doomed Haiku set escalated to Opus recovers 82% at roughly \$1.47 per recovered resolve. Every starting point recovers a majority at a small multiple of the base, so model-tier escalation is a value-of-resolve decision, not a default: worth it whenever a resolved instance is worth more than its \$1.37–\$2.71 escalation cost.

Table 11 and Figure 5 make the rule explicit. Run the base everywhere, since its \$0.82 per resolve is below any plausible value of a resolved bug fix; abandon the agreement-doomed set while the value of a resolve V is below the cheapest escalation’s \$1.37 per recovered resolve; and escalate progressively as V rises past each option’s cost per recovered resolve. The decision is a single comparison of V against a measured per-option threshold, not a tuning sweep.

A core resists escalation entirely, but it is tiny. Only 2/150 instances (both `matplotlib`) are unresolved by every configuration across all three model tiers *and* under an independent agent harness (§12), so there is almost nothing to abandon. The deployable form of the stopping decision is therefore the regression-gated escalation of

Table 11: Value-of-resolve decision rule. Model-tier escalation of the agreement-doomed set pays off when the value V of a resolved instance exceeds that option’s cost per recovered resolve, measured against the \$0.82-per-resolve base operating cost (two-configuration diverse-select, \$9). Reproduces from the logged cells.

Doomed set	Escalate to	Recovers	Escalate iff
Haiku-doomed	Opus (1 config)	40/49 (82%)	$V \geq \$1.47$
Sonnet-doomed	Opus (1 config)	21/27 (78%)	$V \geq \$1.37$
Sonnet-doomed	Opus (2 configs)	22/27 (81%)	$V \geq \$2.71$
3-tier + harness core	any	0/2	never

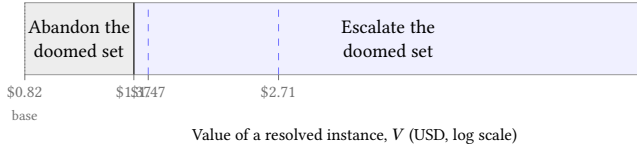


Figure 5: The value-of-resolve decision for the agreement-doomed set, priced in the gold-graded replay (\$9.1). Below the cheapest escalation’s cost per recovered resolve (\$1.37) the set is stopped; above it, model-tier escalation pays off, with the richer options (Table 11) turning on as the value of a resolve rises past their per-option thresholds. The base configuration is run for any value above its \$0.82 per-resolve operating cost. The deployed form of the stopping decision is the regression-gated escalation of \$9.8.

\$9.8 (escalate the flagged set, hand back only the residue), because the single-tier abandon realization fails its integration test (§13.5). Model-tier escalation is the lever; abandonment is not.

The cheaper-model cascade above is the negative; the cross-tier cascade is not. Run end-to-end on the first 150 instances, the cheapest viable cascade (the Sonnet ten-turn signaled-budget cell escalated to the strong tier) Pareto-dominates the best single configuration (the Opus twenty-turn cell, 128 at \$72.03): escalating every failure to that bounded Opus cell reaches 132/150 at \$69.05, more resolves at lower cost, and the win holds under every tie-break ordering of the escalation queue. The regression-gated variant alone (escalate the no-patch failures plus the patch-bearing failures the gate does not clear) reaches 121/150 and does not dominate, because the gate is imperfect; what makes the cascade pay is that a cheap tier-one leaves few failures, so escalating all of them is affordable. The full analysis of this deployable win is in Section 8.4 and Section 9.7.

This first-150 win generalizes to the held-out second-150 split, with two honest qualifications. First, the escalation tier must run *unbounded* on the harder split: the bounded twenty-turn Opus cell that wins on the first 150 is itself dominated by the Sonnet floor on the held-out set, where its turn cap binds on the harder, symbolic-heavy instances. The unbounded Opus tier instead recovers 73% of the escalated failures and lands 124/150, +31 resolved over the Sonnet floor baseline of 93 under the same reference-patch-validated scoring. Second, escalation is costlier on the harder split, so the

cascade buys those +31 resolves at added cost (\$159 in total against the floor’s \$84) rather than for free: on the held-out set it is the quality-max operating point under the value-of-resolve rule above, not a both-axes dominator. The lever generalizes; its tier budget and break-even value are workload-dependent, consistent with the quality-max menu entry being workload-dependent (§8.4). The capability bar that sinks the cheaper-tier cascade is itself vendor-independent: an escalation tier pays only when it out-resolves the cheaper one on the doomed residual, so a model that is merely cheaper per token is not automatically a cheaper escalation tier per resolved instance.

13.2 Warm-starting the escalation does not pay

The cold cascade above re-derives localization and reproduction at every tier, so a natural rescue is a *warm* ladder: escalate cheap-tier failures to the strong tier seeded with the cheap tier’s own artifacts (its reproduction test, touched-file localization, and regression grade). This is honest signal injection, since it uses no benchmark verdict. On the cheap-tier patch-bearing failures it gives no lift. Injecting the prior patch is actively harmful: the strong model treats it as a claim to verify and spends its whole turn budget checking the previous attempt rather than fixing the bug. Injecting only the localization (which files to read) ties a cold escalation on resolves and costs slightly more; an apparent one-instance advantage at $n=12$ washed out to a tie at $n=17$. The result matches the relevance-ranker finding of Section 8.5: a capable model self-locates, so injected context from a cheaper tier is redundant or distracting. The one signal that survives is which cheap-tier failures are worth escalating at all, the close-versus-lost split of Section 9.8.

13.3 Graded model ladder

Beyond the binary cascade we evaluated a graded model ladder: a Haiku-to-Sonnet-to-Opus sequence that ships or steps up per instance on deployable signals, plus a trained rescuability ramp that steps up the instances with the highest predicted rescue probability. No deployable, leak-free ladder reaches the best single configuration’s 128 resolves below its \$72.03 on the first 150. The intermediate tier is dominated: a cheap-tier failure the strong tier can fix, the mid tier usually cannot, so the ladder pays for the mid tier without adding ceiling, and the lowest tier is too weak to serve as a useful first rung (Haiku at a ten-turn signaled budget resolves 17 of 150). The strong tier is the ceiling, which collapses the ladder to the two-tier cascade already analyzed. The menu of single configurations plus a stopping gate, not a graded ladder, is what survives.

13.4 Mid-session signal-driven escalation

A signal-driven mid-session budget escalation (start with a ten-turn budget; when in-session stuck signals fire on file thrashing, repeated test failures, or traceback patterns, extend the budget to twenty-five turns) was evaluated as an alternative to the simpler two-pass critique mechanism (Section 7.1). On Sonnet at first 30 instances the signal-driven cell resolves 17/30 (57%), below the implicit ten-turn cell’s 22/30 (73%) despite the added stuck-signal machinery. The mechanism is Pareto-dominated by simpler budget choices on this workload.

The fixed two-pass critique form (no signals, unconditional reattempt on a quality-gate failure) is the form carried in the variant catalog and the external head-to-head (Section 7.1). The signal-driven form is retained in the shipped catalog for users whose workload has a monomorphic stuck-signal pattern (for example, visualization workloads where the agent gets stuck on rendering-pipeline failures), but it is not on the Pareto frontier at our headline sample size.

13.5 The single-tier run-both controller, retired by its own integration test

An earlier stage of this work composed its three strongest components into one deployable controller: run a diverse pair of configurations (the unbounded baseline plus the signal-injection partner), select between their candidates with a generated fail-on-base reproduction script, and stop early on the instances whose candidates agree on failure. Offline gold-graded replays predicted this composition would beat the external baseline on the cost-quality frontier. We integration-tested it end-to-end, leak-free, on all 300 SWE-bench Lite instances (Section 5.3), against the strongest possible internal control: its own best member run alone.

It failed on three axes.

First, selection adds nothing. Carrying the selected candidate regardless of the stopping decision, the controller resolves exactly what the unbounded baseline resolves alone: 111 versus 111 on the first 150, and 93 versus 93 on the held-out split. The verifier-driven selection added *zero* net resolves across 300 instances.

Second, stopping destroys real resolves. Deployed honestly, with abandoned instances shipping nothing, the controller resolves 84 on the first 150 and 76 held-out. The stopping rule destroys 28 real resolves on the first 150 and 19 held-out.

Third, the cost rises sharply. Deployed honestly it costs roughly \$1.41 per resolve, +124% over the unbounded baseline’s \$0.63. Ship precision is 75% on the first 150.

The arithmetic of Section 9.7 explains why no tuning rescues it. The pair’s routing headroom is +9 instances (an oracle-assisted upper bound) while the pair costs roughly twice a single configuration. The negative replicates on the held-out split.

We report this as a first-class result for two reasons. It is the honest deployment test of the stopping signal of Section 9, and it demonstrates the framework discipline the rest of the paper relies on: a composed controller must beat its own best member, leak-free and end-to-end, before it ships. The retirement is itself conservative: the second member’s generation loop was oracle-assisted (Section 15), so the deployable pair would do no better than measured here.

13.6 Evaluation-gated control flow: a control-channel leakage class

The second methodology negative is a leak in which no individual patch is illegitimate. What leaks is the control flow that decides when to stop or what to retry. The protocol of Section 5.3 now prevents it. In earlier configurations, two control paths consulted the benchmark’s held-out verdict. The two-pass critique variant skipped its second pass when the verdict said the first pass had

already resolved the instance. The signal-injection variant’s generation loop used the verdict as its stop condition and fed failing-test output back into the prompt. Neither path changed what a patch *could* do. The first biased the variant’s measured cost downward. The second inflates every union and ceiling that includes the variant’s cells. The class of bug is easy to acquire and hard to see: any control-flow read of an evaluation signal biases cost downward and resolve counts upward while leaving every individual patch legitimate. The protocol is structural rather than case-by-case: deployable signals (reproduction scripts, regression suites, patch presence) drive all control, and the benchmark verdict scores and nothing else. This protocol ships as the framework default (Section 15).

This is a *control-channel* leak, distinct on mechanism from the data-channel leaks and adversarial exploits catalogued in Section 14.3: the prompt and tests stay clean and every patch is genuine, yet a benign control-flow read of a benchmark-only signal still inflates quality and deflates cost. The same lens flags a leak hiding in standard practice. The oracle@*k* number, routinely reported as a benign best-case ceiling for a pool of *k* candidates, selects the best member by the gold verdict, which is exactly the bit a deployment lacks; read as a deployable figure rather than a ceiling, it is a control-channel leak of the same class. We report deployable selection without it (Section 13.5).

14 Related work

14.1 Agent Capsules

Code-Capsules extends Agent Capsules [30] from multi-agent research-task pipelines to single-session coding agents. Four specific claims from Agent Capsules are extended or refined here. Agent Capsules’ composition-score result (that mechanism rankings transfer cross-model because the score is a behavioral fingerprint rather than a capability ranking) is the conceptual predecessor of our finding that variant rankings transfer cross-vendor on agent benchmarks (Section 10.4). Agent Capsules’ per-model-family behavioral partition (the composition-score fires on Anthropic models but not on the OpenAI or Gemini families) is the cross-vendor analog of our per-model-tier Pareto-frontier characterization (Section 8); the two axes are orthogonal partitions of the same population and yield different deployment guidance. Agent Capsules’ budget-signaling result on research-task pipelines is refined in Section 8.6: on coding tasks under a turn-budget framing, the signaling response is gated by spare reasoning capacity at the budget rather than by model size, the opposite direction of the research-task pattern. Finally, the ranker-effectiveness-inversely-scales-with-capability finding (Section 8.5) is the same mechanism story as Agent Capsules’ Behavioral Stability Index collapse on Haiku under compound framing: a more-capable substrate suppresses the benefit of a planted scaffold. Replicating the shape on a different mechanism (file pre-selection rather than multi-agent framing) suggests it generalizes beyond either specific primitive.

14.2 Coding-agent harnesses

SWE-Agent [40] introduced an agent-computer interface abstraction (a custom shell, file viewer, and edit tools) tailored for

SWE-bench. The three SWE-bench Lite harnesses most-directly positioned as competitors to the configuration framework in this paper are Agentless [39], AutoCodeRover [45], and OpenHands [36]. Agentless [39] is a scaffold-light two-stage retriever + repair pipeline; our matched-model re-run (Section 11) places its cost-quality region precisely relative to the framework’s configuration menu, anchored against the balanced two-pass entry. AutoCodeRover [45] couples LLM reasoning with structure-aware code search (AST-level navigation) and is the closest published analog to the relevance ranker primitive in Section 7.3. OpenHands [36] is the de-facto open-source SWE-bench harness as of writing, with a CodeActAgent loop that an operator would plausibly pick up by default. MetaGPT [16] and AutoGen [38] are general multi-agent frameworks that have also published coding-agent results.

Cited published numbers from these harnesses are not directly comparable to our cells without harness control (model version, prompt template, instance subset, cache-aware versus naive cost accounting, and per-instance compute budget all vary). For the $n=150$ cells reported in this paper we anchor against the binomial standard error band of ± 4 percentage points (Section 5.4) rather than against any single external number. We additionally report a controlled head-to-head re-run of Agentless under our Docker eval pipeline on the full 300-instance set (Section 11); an equivalent OpenHands re-run remains open follow-up work (Section 15). The framework’s contribution is orthogonal to harness design and operates as a runtime layer that could in principle wrap any of these harnesses through a `ModelClient` implementation specific to that harness’s tool surface.

14.3 Benchmark and evaluation integrity

A growing literature documents ways SWE-bench results overstate deployable capability, along two lines. The first is data-channel: the gold solution or its sketch appears in the issue text, or the held-out tests are too weak to reject a wrong patch. SWE-Bench+ [1] finds the solution leaked in the issue for 32.67% of accepted patches, with weak tests admitting another 31.08%; UTBoost [42] strengthens the gold tests and revises 40.9% of SWE-bench Lite leaderboard entries; and the SWE-bench Verified audit [26] re-annotates the suite for underspecified issues and weak tests. The second line is adversarial: the agent exploits the evaluation rather than solving the task. ImpossibleBench [47] measures how often a model takes a specification-violating test shortcut, and CapCode [23] caps achievable scores on randomized tests so that scores above the cap reveal cheating. Closest to this paper, the trace auditor Meerkat [33] and the accompanying audit [34] surface *harness-level* cheating, report a reported-versus-clean pass-rate gap, and note that such cheating is not always intentional.

The leakage class named in Section 13.6 is distinct from both lines on *mechanism*. It is not a data-channel leak: the prompt and tests are clean and every patch is genuine. It is not an adversarial exploit: no agent reads or games the verdict. It is a *control-channel* leak, in which a benign harness design choice lets a benchmark-only bit (the gold pass/fail verdict, `test_patch`, or post-fix state) gate a control-flow decision (stop, retry, escalate, abandon, or select among samples), inflating reported quality and deflating reported

cost by a margin no deployment recovers. We measure that gap directly under the matched leak-free protocol of Section 5.3.

14.4 Test-based selection and verification

Selecting among candidate programs by executing generated tests is an established line. AlphaCode [22] filters and clusters millions of samples through generated test inputs, an approach that repeated-sampling coverage scales further [6]; CodeT [7] generates test cases with the same model that wrote the candidates and selects by dual execution agreement; and Agentless [39] itself reranks SWE-bench patches with model-generated reproduction tests plus regression-test outcomes. The fail-on-base reproduction verifier in Section 9 sits in this lineage and claims no novelty for the mechanism. What this paper adds is the deployment-side measurement the lineage lacks: run leak-free at $n=300$ inside an end-to-end controller, generated-test selection between two strong agentic candidates added zero resolves over a fixed candidate-order baseline (Section 13.5): at current reproduction-test granularity, diverse agentic patches saturate the suite and selection cannot discriminate between them. The regression-suite signal is used differently here as well: not as a reranker but as the gate that rescues wrongly-abandoned instances and targets escalation (Section 9.6, §9.8), with its independence from the held-out evaluation measured explicitly.

14.5 Retrieval-augmented coding agents

RepoCoder [44] uses iterative retrieval at session time to surface relevant code for the model to read. The relevance ranker in Section 7.3 is structurally simpler (a single BM25 [31] pass at session start with a static prepend) and operates at a different point in the cost-quality tradeoff: cheaper per session, less adaptive within session. The two approaches are orthogonal; a future `ModelClient` implementation could expose runtime retrieval as a tool the agent invokes inside its loop, composing with our shipped variants.

14.6 Cost-quality framings

FrugalGPT [8] and RouteLLM [25] address cost-quality routing in the LLM-routing setting (which model to send a request to). Abstention-based cascades that defer or decline rather than always escalate [11, 43], and input-adaptive compute allocation [10], share the stopping intuition; our cheaper-model cascade (Section 13.1) reports the negative deployment result for that family on agentic SWE-bench patches, where the deployable distress signals do not separate the recoverable tail. Two results frame why no governor on this signal pays. Bouchard [5] shows that the structural cost of running the cheap tier on every instance, not routing quality, is what bounds cascade savings, which is the boundary our governor sits against (Section 13.1); and Pugnana and Ruggieri [29] show that a signal’s detection quality (its area-under-curve) is decoupled from its value for a selective decision, which is why a label-free signal can be a strong doom detector (Section 9) and still fund no profitable policy. The routing primitive in Section 6.3 is the analog in the agent-loop setting (which shipped variant to use for a given workload class). The two layers compose: a cost-aware LLM router could front a Code-Capsules deployment, dispatching to a cheaper agent runtime for tasks where the cost-quality knee favors it.

Kapoor et al. [20] argue that agent evaluation should jointly optimize cost and accuracy and report Pareto frontiers rather than accuracy alone; that critique motivates the framing here, and this paper does not claim it. What the menu in Section 4 adds to that program is a worked instance with the accounting hardened: a single modeled cost surface across all systems, leak-free control flow (Section 13.6), and a held-out split under the canonical evaluator, together with the negative result that a plausible adaptive controller loses to a point on the menu once oracle assistance is removed (Section 13.5).

14.7 Prompt compilation and compression

DSPy [21] compiles prompts and pipelines at design time; LLM-Lingua [17] compresses individual prompts to reduce token count. Code-Capsules operates orthogonally: variant selection and mid-session adaptation happen at runtime against the deployed model, without a training corpus or a design-time compilation step. A DSPy-compiled prompt could in principle be served through Code-Capsules’ shipped variants; we have not measured this combination.

14.8 LLM-as-judge evaluation

Our quality gate on SWE-bench Lite is the test-patch outcome under Docker (Section 5.1), not a learned judge. For workloads without an executable quality signal, the framework’s quality-gate primitive supports user-supplied judges in the style of MT-Bench [46]; the shipped implementations cover Docker-based evaluation for SWE-bench, pytest-based evaluation for HumanEval and MBPP, and an always-pass stub for workloads under user-supplied gates.

15 Limitations

Vendor and harness coverage. Code-Capsules ships ModelClient implementations for the Anthropic, OpenAI, and Google vendor families. Within Anthropic we drive Claude through the Claude Code CLI; within OpenAI we drive both Chat Completions and the Responses API; within Google we drive Gemini 2.5-flash. We report a controlled head-to-head re-run of Agentless [39] under our evaluation pipeline (Section 11); we do not re-run SWE-Agent [40], AutoCodeRover [45], or OpenHands [36]. The cross-vendor primitive supports such comparisons in principle, and an OpenHands re-run under our Docker pipeline is the immediate next phase of this work.

Programming-language scope. All three benchmarks are Python. The shipped variants encode no Python-specific assumptions, but the empirical calibrations do not establish the shipped defaults on non-Python codebases. Extension to SWE-bench-Multilingual is open follow-up.

Sample-size scope. SWE-bench Lite headline cells are at $n=150$ across all four model families (Anthropic Haiku 4.5, Sonnet 4.6, Opus 4.7, and OpenAI gpt-5-codex); HumanEval is at $n=164$; MBPP is at $n=500$. The SWE-bench Lite sample is the benchmark’s first 150 instances of 300; the remaining 150 (held-out set, dominated by sympy) is a measured held-out surface for the unbounded baseline, the stopping controller and its regression-suite gate, the two-pass critique cell, and the Agentless head-to-head (Sections 9, 11). Per-tier headline cells for the other variants remain at $n=150$; in

particular the Opus twenty-turn quality-max cell has no held-out replication yet, so its menu position rests on the calibration split.

Cost-accounting scope. Cost figures use vendor public pricing as of the pricing date stamped on each shipped cost model (Section 5.6), with cache-aware accounting for the vendor cache discount. Batch-API discounts (approximately fifty percent off output tokens for asynchronous batched requests on Anthropic and OpenAI) and enterprise or committed-use pricing tiers are not modeled. Production deployments with either should subclass the shipped cost models to honor their negotiated rates.

Statistical and interpretive scope. We report observed pass rates and per-cell costs without formal confidence intervals; per-cell binomial standard error is noted in Section 5.4. The capability-headroom-mediated budget-signaling hypothesis (Section 8.6) is consistent with the cross-tier evidence at $n=30$ per cell but is not a confirmed law. The cross-vendor variant-transfer findings are evaluated on two vendors (Anthropic and OpenAI); transfer to vendor families beyond those (Mistral, open-weight model families, additional hyperscaler offerings) is not measured.

Stopping-signal and gate scope. The agreement signal and its gates (§9) hold under the following scope. Two of these are deployer choices, not caveats: pick the partner configuration per tier, and validate the regression-suite gate on your own workload before trusting it.

The best partner configuration is tier-specific. The signal-injection variant partners well at the mid and strong tiers but collapses on the weakest model, where a budget-lever configuration partners better, so a cross-model deployment selects the partner per tier. We report a roughly 22% independent doomed-instance precision, measured across model and harness: most of the flagged set is recoverable at a stronger tier, so the signal is an escalation trigger rather than an abandon verdict. The 90.9% in-family figure is an in-sample upper bound. Deployed precision is bounded by the reproduction verifier, whose 9.1% per-candidate reproduction false-negative accounts for the 28 false-positive ships on the first half. The regression-suite rule was calibrated on django-heavy abandons and discriminates less sharply off django; a gate-negative reading is not doom, since most of the gate-negative set is recoverable at a stronger tier, so decline-by-default arms need per-workload justification. Controller and escalation dollar figures are anchored to the first 150 mid-tier instances and, where a held-out cost was not measured directly, are modeled from calibration-split accounting; the strong-tier escalation figure is a single run. Resolve counts are measured and qualitative conclusions hold across tiers, but exact dollars shift with the slice, and precision and flag-count figures are single-run and carry the per-cell variance of Section 5.4.

Evaluation-protocol conditions. Two conditions bound specific reported numbers and are noted wherever those numbers appear. The signal-injection variant’s generation loop read the held-out evaluation verdict as its stop condition, so every cell of that variant, and any union or ceiling containing it, is oracle-assisted; this includes the 148/150 reachability union (Section 12) and the +9 pair-routing headroom (Section 9.7), and the controller retirement of Section 13.5 is conservative under it. Separately, solver prompts

include the benchmark’s target-test identifiers, which the issue-only Agentless baseline does not receive, an asymmetry in our favor across the Section 11 comparison that is partly offset by the oracle@ k handicap running the other way. The harness exposes a flag to disable target-test inclusion; a bounding re-run is open work.

Controller–harness boundary. The framework owns the controller decision logic (round orchestration, the diverse-sample agreement test, and the early-termination and model-tier-escalation triggers), together with the mid-session prompt generation for every shipped configuration, including the signal-injection partner’s resume prompts. What remains in the benchmark harness is SWE-bench-specific input/output (Docker evaluation, repository checkout, dataset loading); it re-exports the framework’s prompt builders rather than defining its own. The integration-tested two-configuration controller is therefore reproducible from the shipped runtime, not only from the benchmark harness.

16 Discussion and conclusion

The deployable menu. The framework’s primary value is the menu of configurations with calibrated defaults rather than any single mechanism in isolation. The shipped `policy.yaml` (configuration surface described in Section 4) encodes the measured menu of Table 2: the relevance ranker at a ten-turn signaled budget for cost minimization (\$0.34 per resolve); two-pass critique as the balanced entry anchored against the external baseline; the unbounded Sonnet baseline as the mid-tier quality pick; and the Opus twenty-turn implicit cell as quality-max, which resolves more than the unbounded Sonnet baseline at lower cost per resolve, with no relevance-ranker stacking (the ranker regresses on Opus by -6 to -17 percentage points across variants). On the cheapest tier, Haiku stacked with the relevance ranker plus plan-then-execute is the cost-minimization pick (the only tier where the ranker composes positively with another variant). Users in other workloads re-derive their own frontier from per-cell evaluation (Section 4.2).

The extension surface as the reusable artifact. The headline result is the mechanism plus the discipline that tested it; the eight extension primitives in Section 3 are how both port: what a deployer in a different domain actually re-uses is the mechanism portfolio and the measurement loop, not the SWE-bench Lite calibration per se. The shipped defaults are evidence of feasibility (the primitives reach a deployable Pareto frontier on a real benchmark across three model tiers and three vendors), and the same surface is what lets a new workload re-derive its own frontier, its own menu, and its own gate thresholds. The cross-vendor primitive in particular is the single largest delta from Agent Capsules [30] in this paper: it makes per-workload, per-vendor cost-quality measurement reproducible on the same orchestration code, which makes the per-workload Pareto frontier a first-class object rather than an artifact of any specific vendor’s harness.

Cross-vendor as validation. The cross-vendor variant ladder (Section 10.3) provides one form of mechanism validation. Plan-then-execute lifts pass rate on both Sonnet and gpt-5-codex (Sonnet 56.0 $\rightarrow$ 62.0%; codex 30.7 $\rightarrow$ 32.7%), which is evidence that the

data-axis scaffold operates above any specific vendor’s model architecture. Two-pass critique lifts both vendors as well: the always-run second pass resolves 98 leak-free against plan-then-execute’s 93 on Sonnet, and a mean of 66.7 against 49 on codex. The codex gain comes despite its dominant failure mode being an empty patch (no `<patch>` block emitted) on roughly 44% of instances: the always-run second attempt recovers enough of the remainder to move the cell, though the empty-patch population caps codex’s ceiling well below Sonnet’s. Because that empty-patch population flips run-to-run on the same instance (Section 10.4), we report it as the observed failure mode of our shipped configuration rather than an intrinsic capacity limit. Both transferring and non-transferring mechanisms inform deployment: the framework reports both as per-vendor cells rather than promoting any single mechanism unconditionally.

Future work. Three directions are immediate. Multi-language workloads: SWE-bench is Python-only, and the relevance ranker’s BM25 implementation does not handle multi-language repositories without configuration. Head-to-head comparisons against SWE-Agent [40] and other harnesses under our evaluation pipeline. Porting the relevance-ranker primitive back to Agent Capsules’ research-task workloads, where issue-relevant document selection has the same structural shape as issue-relevant file selection. We also note the workload capability ceiling (Section 12) as an open mechanism question: ensembling, pass-at- k aggregation across independent sessions, and multi-shot self-debug protocols are candidates that might unlock instances no shipped variant currently resolves.

Central finding. The most consequential finding on SWE-bench Lite is that a menu of single configurations plus a regression-gated escalation gate dominates the adaptive single-tier run-both controller on cost at held quality. Against the single external baseline we re-ran, it resolves more at 13.8% lower cost per resolve, a significant quality edge ($p = 0.0055$): a win on both axes. The diverse-sample agreement signal flags hard instances at a tier (the best in-family pair fires on 22 and false-abandons 2), but its single-tier deployment (run a diverse pair, select with a verifier, abandon on agreed failure) was retired by its own leak-free integration test: zero resolves added over its best member, 28 honest resolves destroyed per 150 instances, 124% higher cost per resolve (Section 13.5). Knowing when to stop matters at the *tier boundary*, not inside one tier: the regression-suite gate ships the candidates the verifier wrongly discarded at zero added cost, recovering most of them (84 to 99, 12 short of the unbounded baseline’s 111); the flagged set is mostly recoverable at the next tier, where the strong tier converts 21 of the 21 gate-positive instances escalated to it and 15 of the 17 gate-negative. The secondary lever is how the per-instance budget is spent: plan-then-execute, the relevance ranker, and the model-tier choice each buy a measured position on the cost-quality frontier, and the strong-tier twenty-turn cell shows that stepping up a tier at a bounded budget can beat unbounded turns at the tier below on both quality and cost per resolve. The framework’s job is to make all of these choices measurable (when to stop, when to escalate, and how to spend what remains) per workload, per vendor, and per model tier, to ship the calibrated defaults as a deployable policy file, and to integration-test any composed controller against its own best member before shipping it.

References

- [1] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. *arXiv:2410.06992*.
- [2] Anthropic. 2024. Prompt Caching with the Anthropic API. <https://platform.claude.com/docs/en/docs/build-with-claude/prompt-caching>. Developer documentation.
- [3] Anthropic. 2025. Claude Code: Anthropic’s Coding-Agent CLI. <https://code.claude.com/docs/en/overview>. Product documentation.
- [4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021). <https://arxiv.org/abs/2108.07732>
- [5] Dylan Bouchard. 2026. Is Escalation Worth It? A Decision-Theoretic Characterization of LLM Cascades. *arXiv:2605.06350*.
- [6] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. *arXiv:2407.21787*.
- [7] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zhan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT5: Code Generation with Generated Tests. In *International Conference on Learning Representations (ICLR)*. *arXiv:2207.10397*.
- [8] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023). <https://arxiv.org/abs/2305.05176>
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). <https://arxiv.org/abs/2107.03374>
- [10] Mehul Damani, Idan Shenfeld, Andi Peng, Andreea Bobu, and Jacob Andreas. 2025. Learning How Hard to Think: Input-Adaptive Allocation of LLM Computation. In *International Conference on Learning Representations (ICLR)*. *arXiv:2410.04707*.
- [11] Hen Davidov, Nachshon Cohen, Oren Kalinsky, Yaron Fairstein, Guy Kushilevitz, Ram Yazdi, and Patrick Rebeschini. 2026. Knowing When to Quit: A Principled Framework for Dynamic Abstention in LLM Reasoning. In *Proceedings of the 43rd International Conference on Machine Learning (ICML) (PMLR, Vol. 306)*. *arXiv:2604.18419*.
- [12] Google. 2025. Gemini API: Function Calling and Context Caching. <https://ai.google.dev/gemini-api/docs>. Developer documentation.
- [13] Matt Gorbett and Suman Jana. 2026. Cross-Model Disagreement as a Label-Free Correctness Signal. *arXiv:2603.25450*.
- [14] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujia Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations (ICLR)*. *arXiv:2305.11738*.
- [15] Kimia Hamidieh, Veronika Thost, Walter Gerych, Mikhail Yurochkin, and Marzyeh Ghassemi. 2026. Complementing Self-Consistency with Cross-Model Disagreement for Uncertainty Quantification. In *International Conference on Learning Representations (ICLR)*. *arXiv:2604.17112*.
- [16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*. *arXiv:2308.00352*.
- [17] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. *arXiv:2310.05736*.
- [18] Carlos E. Jimenez, John Yang, and Jiayi Geng. 2024. SWE-bench Lite: A Curated Subset of SWE-bench for Efficient Evaluation. <https://www.swebench.com/lite.html>. Dataset note; cite [19] for the underlying benchmark.
- [19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. *arXiv:2310.06770*.
- [20] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2025. AI Agents That Matter. *Transactions on Machine Learning Research* (2025). *arXiv:2407.01502*.
- [21] Omar Khattab, Arnab Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations (ICLR)*. *arXiv:2310.03714*.
- [22] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-Level Code Generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. *arXiv:2203.07814*.
- [23] Thanawat Lodkaew, Johannes Ackermann, Soichiro Nishimori, Nontawat Charoenphakdee, Masashi Sugiyama, and Takashi Ishida. 2026. Do Coding Agents Deceive Us? Detecting and Preventing Cheating via Capped Evaluation with Randomized Tests. *arXiv:2606.07379*.
- [24] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*. *arXiv:2303.17651*.
- [25] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs from Preference Data. In *International Conference on Learning Representations (ICLR)*. *arXiv:2406.18665*.
- [26] OpenAI. 2024. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>.
- [27] OpenAI. 2024. Prompt Caching in the OpenAI API. <https://developers.openai.com/api/docs/guides/prompt-caching>. Developer documentation; automatic for prompts ≥ 1024 tokens; cached input billed at 10% of fresh rate on the GPT-5 family.
- [28] OpenAI. 2025. Responses API. <https://platform.openai.com/docs/api-reference/responses>. Developer documentation.
- [29] Andrea Pugnana and Salvatore Ruggieri. 2023. AUC-based Selective Classification. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (AISTATS) (PMLR, Vol. 206)*. 2494–2514. *arXiv:2210.10703*.
- [30] Aninda Ray. 2026. Agent Capsules: Quality-Gated Granularity Control for Multi-Agent LLM Pipelines. *arXiv preprint arXiv:2605.00410* (2026). <https://arxiv.org/abs/2605.00410>
- [31] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.
- [32] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*. *arXiv:2303.11366*.
- [33] Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. 2026. Detecting Safety Violations Across Many Agent Traces. *arXiv:2604.11806*.
- [34] Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. 2026. Finding Widespread Cheating on Popular Agent Benchmarks. <https://debugml.github.io/cheating-agents/>.
- [35] Lei Wang, Wanyu Xu, Yihui Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. *arXiv:2305.04091*.
- [36] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations (ICLR)*. *arXiv:2407.16741*.
- [37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*. *arXiv:2203.11171*.
- [38] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Conference on Language Modeling (COLM)*. *arXiv:2308.08155*.
- [39] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Agentless: Demystifying LLM-based Software Engineering Agents. In *Proceedings of the ACM on Software Engineering (PACMSE)*, Vol. 2, FSE. 801–824. doi:10.1145/3715754 *arXiv:2407.01489*.
- [40] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*. *arXiv:2405.15793*.
- [41] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. *arXiv:2210.03629*.
- [42] Boxi Yu, Yuxuan Zhu, Pinjia He, and Daniel Kang. 2025. UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. *arXiv:2506.09289*.
- [43] Michael J. Zellinger, Rex Liu, and Matt Thomson. 2025. Cost-Saving LLM Cascades with Early Abstention. *arXiv:2502.09054*.

- [44] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2303.12570..
- [45] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. arXiv:2404.05427..
- [46] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2306.05685..
- [47] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. 2026. ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. In *International Conference on Learning Representations (ICLR)*. arXiv:2510.20270..